

Tricycle: Private Transformer Inference with Tricyclic Encodings

Lawrence Lim

University of California, Santa Barbara
lawrencecklim@ucsb.edu

Divyakant Agrawal

University of California, Santa Barbara
divyagrawal@ucsb.edu

Vikas Kalagi

University of California, Santa Barbara
vikaskalagi@ucsb.edu

Amr El Abbadi

University of California, Santa Barbara
elabbadi@ucsb.edu

ABSTRACT

The growing adoption of Large Language Models in privacy-sensitive domains necessitates secure inference mechanisms that preserve data confidentiality. Homomorphic encryption offers a promising pathway by enabling computation on encrypted inputs, yet existing approaches struggle to scale efficiently to full transformer models due to limitations in packing schemes, which must efficiently support a wide range of operations, including matrix multiplications, row-wise nonlinear operations, and self-attention. In this work, we present Tricycle, a framework for private transformer inference built on our novel packing scheme, called tricyclic encodings, which are designed to efficiently support these core operations. Tricyclic encodings are a generalization of bicyclic encodings, enabling privacy-preserving batch matrix multiplications with optimal multiplicative depth in order to facilitate parallelized multi-head self-attention. We optimize our matrix multiplications by incorporating Baby-Step Giant-Step optimizations to reduce ciphertext rotations and presenting new ciphertext-plaintext matrix multiplication techniques that relax prior limitations. A further contribution of our work is a lightweight and effective approach for stabilizing the softmax function via statistical max estimation. Our end-to-end implementation on a BERT-Tiny model shows that Tricycle achieves a 1.5 $\times$ to 3 $\times$ speedup over previous approaches, marking a step toward practical and scalable private LLM inference without sacrificing model fidelity.

1 INTRODUCTION

Through the invention of transformer models, Large Language Models (LLMs) have unlocked a wide range of innovative applications thanks to their powerful text processing capabilities. However, a significant barrier to broader adoption is the need to query these models with sensitive or private data. For example, healthcare professionals may wish to use an LLM to summarize a patient’s clinical history, while educators might want help grading student exams more efficiently. Both involve query data that require confidentiality. Even everyday users may seek the benefits of LLMs without compromising their privacy. Despite the potential benefits of private LLM inference in these scenarios, model owners who train LLMs using vast computational and data resources are often reluctant to enable private model execution by sharing their model weights with users, as doing so could erode their competitive advantage.

One solution is to use homomorphic encryption to enable private inference on LLMs. Homomorphic encryption allows computations to be performed directly on encrypted data, preserving privacy

throughout the entire process. This approach allows a user to encrypt their query and send it to the model owner without revealing any sensitive information. Then, the model owner homomorphically evaluates the LLM, composed of matrix multiplications and nonlinear operations, on the encrypted input and returns an encrypted result, which only the user can decrypt to obtain the output.

Private LLM inference with homomorphic encryption has recently received research interest in a number of research papers [9, 21, 23–26]. A central challenge in scaling these methods to support large inputs and full-scale LLMs lies in the *multiplicative depth* of the computations: higher multiplicative depth requires larger cryptographic parameter sizes, increases ciphertext noise growth, and necessitates more frequent and expensive bootstrapping operations. Prior works have reduced multiplicative depth by replacing specific transformer components, such as softmax [24], layer normalization [25], or both [23], with homomorphic encryption-friendly approximations. However, a key limitation of these works is that the matrix multiplication algorithms they employ often have suboptimal multiplicative depth, which can significantly degrade overall performance. This is particularly problematic because each transformer layer involves eight matrix multiplications, and even a single extra level of multiplicative depth per matrix multiplication can result in an additional expensive bootstrapping operation per layer, compounding the overhead across layers.

Achieving optimal multiplicative depth for matrix multiplications is nontrivial, as they must be integrated into a broader system where other operations, such as matrix transposition, row-wise softmax, and layer normalization, are efficiently supported by the organization of data in ciphertexts, known as *packing*. A carefully designed packing strategy reduces multiplicative depth by avoiding costly reorganization steps, while also improving performance by packing more data into fewer ciphertexts and exploiting the Single Instruction, Multiple Data (SIMD) parallelism inherent to homomorphic encryption schemes.

In this paper, we introduce Tricycle, a novel system for private LLM inference. Tricycle is built on our new packing scheme, called *tricyclic encodings*, enabling *batch matrix multiplication* algorithms with optimal multiplicative depth. Tricyclic encodings generalize prior work on bicyclic encodings [3] and we even improve over prior work by offering both significant performance gains and new ciphertext-plaintext matrix multiplication algorithms with fewer limitations. Crucially, tricyclic encodings are tailored to fit in a broader private transformer inference system: Tricycle solves many of the key packing challenges in enabling private LLM inference. Tricycle operates on generic, pretrained LLMs without requiring

any retraining, leveraging polynomial approximations to implement nonlinear functions. To further support private transformer inference, we present a lightweight and effective technique for stabilizing the softmax function under homomorphic encryption, which is critical for attention computations. We provide an end-to-end implementation of Tricycle, demonstrating $1.5\times$ to $3\times$ speedups over state-of-the-art CPU-based private Bert-Tiny inference systems.

The remainder of the paper is organized as follows: Section 2 provides background information on homomorphic encryption. Section 3 discusses the challenges of enabling private LLM inference and prior work. Section 4 describes recent work on bicyclic encodings [3]. In Section 5, we generalize bicyclic encodings with our novel packing scheme, *tricyclic encodings*, and several new batch matrix multiplication algorithms. Section 6 discusses our improvements to the matrix multiplication algorithms. Section 7 describes how we approximate nonlinear layers and our novel method of stabilizing the softmax. Section 8 details Tricycle’s protocol. Section 9 features an evaluation of our end-to-end private LLM system, Tricycle. Finally, we conclude our paper in Section 10.

2 HOMOMORPHIC ENCRYPTION

Homomorphic encryption enables computation on encrypted data, allowing sensitive information to remain private while still being processed for machine learning without exposing the raw data. We use CKKS [7], an approximate homomorphic encryption scheme that operates on floating-point data. In CKKS, plaintexts are encoded as elements of a polynomial ring $R = \mathbb{Z}[X]/(X^N + 1)$ where N is a large power of two (e.g. $N = 2^{16}$). Each plaintext encodes a vector of real numbers, with the number of slots s equal to $N/2$. CKKS supports several operations:

- **KEYGEN**(1^λ): Given a security parameter λ , generates a secret key sk and the corresponding public key pk .
- **ENCRYPT**(pk, m): Encrypts a vector of floating points m using the public key pk , yielding a ciphertext.
- **DECRYPT**(sk, c): Decrypts a ciphertext c using the secret key sk to recover the approximate plaintext.
- **ADD**(c, x): Adds two ciphertexts or a ciphertext and plaintext c and x element-wise, producing a ciphertext $c + x$.
- **MULTIPLY**(c, x): Multiplies two ciphertexts or a ciphertext and plaintext c and x element-wise, producing a ciphertext that encrypts $c \cdot x$. We denote this as $c \odot x$.
- **ROTATE**(c, i): Rotates the ciphertext c by i positions to the left in a circular fashion. We denote the rotation operation with ρ , so $\rho(c, i)$ for a ciphertext c encrypting $(a_1, a_2, \dots, a_s)$ becomes a ciphertext encrypting $(a_i, a_{i+1}, \dots, a_s, a_1, a_2, \dots, a_{i-1})$.

The efficient use of homomorphic encryption algorithms like CKKS mainly depends on two factors: minimizing the multiplicative depth of computations and reducing the number of operations. *Multiplicative depth* refers to the number of sequential multiplications applied to a ciphertext. Each time a ciphertext undergoes a MULTIPLY operation, its multiplicative depth increases. After a certain depth is reached, a bootstrapping operation is required to enable further multiplication operations. Because bootstrapping is computationally intensive, it is essential to minimize the multiplicative depth of any homomorphic computation whenever possible. In addition

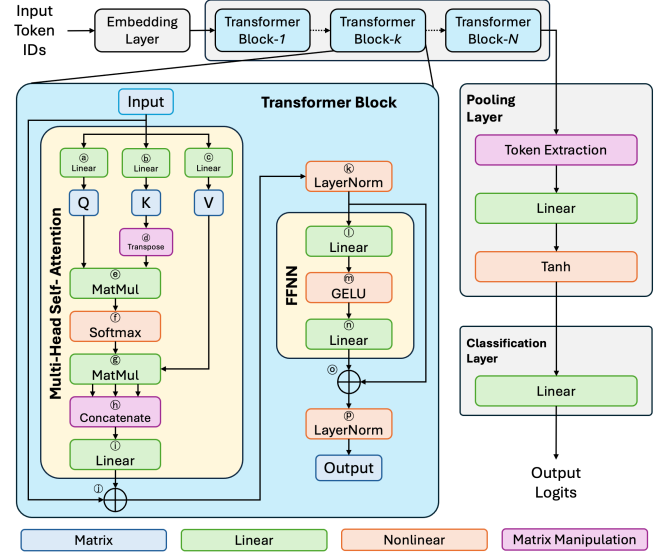


Figure 1: A BERT Model [10]. This figure is partially adapted from Figure 1 in [18].

to multiplicative depth, minimizing the overall number of operations, especially the more expensive key switching operations of rotations and ciphertext-ciphertext multiplications, is crucial for performance. Leveraging the Single Instruction Multiple Data (SIMD) programming model through the element-wise operations of ADD and MULTIPLY helps reduce the number of operations, improving efficiency.

Reducing the overhead in homomorphic encryption depends on how values are assigned to specific slots, a process known as the *packing of a ciphertext*. Optimized packing strategies reduce the computational cost of composite operations, such as matrix multiplications, by enabling more effective SIMD utilization and by lowering both the total number of operations and the multiplicative depth. Additionally, well-designed packing schemes minimize the number of ciphertexts needed to represent a dataset and help avoid costly packing reorganizations (comprised of masking and rotation operations), which in turn reduces the frequency of costly bootstrapping operations. In this paper, we introduce *tricyclic encodings* as a novel packing strategy for enabling the private inference of LLMs. We discuss this more in Section 5.

3 PRIVATE LLM INFERENCE

An LLM typically consists of an embedding layer, followed by a series of transformer blocks and one or more output layers. Figure 1 illustrates an example of such a model: BERT [10], in which the output layers include a pooling layer and a classification layer.

3.1 Challenges

The operations of the transformer model are summarized in Li *et al.*’s survey on private LLM inference [18]. Hence, we now focus on the numerous challenges of enabling such inference using homomorphic encryption. Following the common assumption made by Roviola *et al.* [25], Powerformer [23], and Thor [21], we assume that the user computes the embedding layer in plaintext to generate

token embeddings. As such, we omit discussion of the embedding layer and concentrate on the remaining computations within the transformer. Even with this assumption, enabling private inference remains highly non-trivial due to several key challenges, which relate to the core encrypted computations illustrated in Figure 1 (we use a circled letter, e.g. (a), to refer to specific transformer operations in Figure 1):

- **Matrix Multiplications:** Both ciphertext–plaintext and ciphertext–ciphertext matrix multiplications are needed. For example, the attention mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

requires two ciphertext–ciphertext matrix multiplications: one for computing $Q \cdot K^T$ (e) and another for multiplying the softmax output by V (g). All other matrix multiplications in a transformer are ciphertext–plaintext (a) (b) (c) (i) (l) (n). Besides the performance costs of matrix multiplications under homomorphic encryption, performing successive matrix multiplications is challenging because these algorithms often require specific input packings that may result in another output packing that is incompatible with the next matrix multiplication. Ciphertext–ciphertext matrix multiplications are especially difficult because they require two specific input packings and often result in a third output packing.

- **Multiple Attention Heads:** The computation of attention is divided into h parallel heads, each computing attention independently (a)–(g). The outputs of these heads are then concatenated (h) and multiplied by an attention output matrix (i). A major challenge is in developing a packing scheme that supports these operations while leveraging the SIMD properties of CKKS to parallelize *all* of the operations in the attention heads.
- **Matrix Transposition:** The attention mechanism also requires transposing the matrix K (d). Under homomorphic encryption, transposing can necessitate changing the packing of a ciphertext, which is computationally expensive.
- **Packing Consistency of Residual Connections:** Each transformer block has two residual connections (j) (o) prior to the layer normalization steps. This requires that the packing of the data entering and exiting attention and the feedforward network to remain consistent.
- **Nonlinear Approximations:** The transformer incorporates several nonlinear layers (e.g., softmax (f), layer normalization (k) (o), GELU (m), and tanh) that cannot be directly implemented using only homomorphic addition and multiplication and hence need to be approximated.
- **Row-wise Operations:** Both softmax (f) and layer normalization (k) (o) operate row-wise on their input matrices. Therefore, an effective packing must support efficient row-wise operations.
- **Ciphertext Capacity Limits:** For larger models or when processing many input tokens, it becomes *impossible* to pack entire matrices into the limited slots available in a single ciphertext, even with larger ring dimensions such as $N = 2^{16}$.
- **Bootstrapping Requirement:** Due to the high number of operations in a transformer model, a non-interactive system must

Table 1: Comparison of Matrix Multiplications in Private LLM Inference

Work	Packing	Ctxt-Ptxt Matmul Depth	Ctxt-Ctxt Matmul Depth
Rovida <i>et al.</i> [25]	row-major	2	2
NEXUS ¹ [26]	row, column, diagonal-major	≥ 1	≥ 2
Rho <i>et al.</i> [24]	row-major	3	3
Powerformer ² [23]	column-major	2-3	3-4
Thor ² [21]	diagonal-major	2-3	2-3
Tricycle (ours)	tricyclic encoding	1	1

incorporate bootstrapping to enable more multiplication operations after a certain multiplicative depth has been reached. To minimize this overhead, efficient packing schemes paired with algorithms that minimize multiplicative depth is required.

Needless to say, developing an end-to-end private LLM inference system such as Tricycle is a complex endeavor that demands overcoming all of these challenges at once.

3.2 Prior Work

Private inference of LLMs using cryptography comes in two main categories: interactive, multiparty computation-based approaches and non-interactive, homomorphic encryption-based approaches. In interactive approaches, homomorphic encryption is often used to compute linear layers (such as matrix multiplications), while MPC protocols handle nonlinear layers (such as softmax). Several works have explored interactive protocols for private transformer inference [13, 14, 19, 20, 22, 27]; however, these methods require multiple rounds of communication during inference, which can increase latency and communication overhead. In contrast, our work focuses on the non-interactive approach, leveraging homomorphic encryption exclusively. Several recent works tackle private LLM inference under the non-interactive setting [9, 21, 23–26], with Rho *et al.* also confronting private LLM training [24].

These works explore different matrix multiplication algorithms under homomorphic encryption, as summarized in Table 1, which highlights the packing strategies and multiplicative depth for both ciphertext–plaintext and ciphertext–ciphertext matrix multiplications. Rovida *et al.* [25] introduce algorithms for repeated, column-major packings (VECMATRC) and expanded, row-major packings (VECMATER). NEXUS [26] develops a row-major packing algorithm amortized over multiple input matrices, while also using column-major and diagonal-major packings. However, Thor [21] criticizes NEXUS for insufficiently explaining these matrix multiplication and packing methods. We also note that NEXUS’s open-source code lacks an end-to-end implementation. Both Powerformer [23] and Rho *et al.* [24] build on matrix multiplication algorithms by Jiang *et al.* [15]. While Powerformer extends these algorithms, Rho *et al.* reduces the ciphertext–ciphertext multiplications during private training. Thor [21] introduces matrix multiplications using

¹NEXUS [26] often yields misaligned packings, requiring an unspecified amount of additional multiplicative depth to correct.

²Powerformer [23] and Thor [21] use complexification to reduce operations at the cost of higher multiplicative depth; the ranges reflect with or without complexification.

diagonal-major packings, extending them to support parallel matrix multiplications. Despite these efforts, Table 1 shows that these approaches incur a suboptimal multiplicative depth greater than one, leading to costly bootstrapping operations in deep networks.

These works also offer diverse approaches for the nonlinear layers. Some replace nonlinear operations with polynomial approximations: for instance, NEXUS [26] provides several polynomial approximations for nonlinearities, which Castro *et al.* [9] later utilize. Roviada *et al.* [25] use Chebyshev approximations [4] for operations such as GELU and inverse. Other works modify nonlinear operations to better suit homomorphic encryption. Roviada *et al.* [25] estimate layer normalization using training-derived constants. Rho *et al.* [24] propose Gaussian kernel attention, a variant of attention that eliminates the inverse in softmax, simplifying the homomorphic encryption operations. Powerformer [23] offers homomorphic encryption-friendly alternatives to layer normalization and softmax, using model distillation to mimic plaintext models. A major drawback of modifying nonlinear operations in this way is that they often entail retraining and can degrade model accuracy.

3.3 Our Contributions

Our goal is to provide end-to-end private inference of transformer models. A major limitation of prior work is that the matrix multiplication algorithms introduce extra multiplicative depth, which affect the practicality of these systems. We aim to bridge this gap by improving and building on prior work on bicyclic encodings [3], which proposed generic privacy-preserving matrix multiplications with *optimal* multiplicative depth matrix multiplications. Our work improves on bicyclic encodings by enabling *batch matrix multiplications*, which is needed to parallelize multi-head self-attention, minimizing limitations on ciphertext-plaintext matrix multiplications (such as the sizes of matrices packed into a ciphertext), and significantly improving performance. To enable the private inference of generic LLMs, we avoid modifying the transformer to require retraining by using polynomial approximations of the nonlinear functions. We also propose a novel, efficient method to stabilize softmax through statistical max estimation. All of these techniques are used in our larger private LLM inference system, Tricycle.

4 BICYCLIC MATRIX MULTIPLICATIONS

A central research question in the development of private inference of transformers and LLMs is how to efficiently pack values into a ciphertext while enabling matrix multiplications and nonlinear operations. To address this challenge, we build on the recent work of Chen *et al.* on bicyclic matrix multiplication algorithms (BMM-I and BMM-II) for homomorphic encryption [3]. In this section, we introduce the concept of bicyclic encodings and focus primarily on BMM-I which supports larger matrix packings at the cost of more operations than BMM-II. In Section 5, we present a generalization, called a *tricyclic encoding*, which is designed to enable batch matrix multiplications. Then, in Section 6, we propose optimizations to further improve the efficiency of our algorithms.

4.1 Bicyclic Encoding

Let $m, n \in \mathbb{N}$ with $\gcd(m, n) = 1$. Consider the matrix

$$A = [a_{i,j}] \in \mathbb{R}^{m \times n}. \quad (2)$$

The *bicyclic encoding* of A is defined by the mapping

$$\varphi: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \cdot n},$$

given by

$$\varphi(A)_k = a_{k \bmod m, k \bmod n}, \quad \text{for all } k \in \mathbb{Z}_{mn}, \quad (3)$$

where for each k the expressions $k \bmod m$ and $k \bmod n$ denote the unique remainders in $\{0, 1, \dots, m-1\}$ and $\{0, 1, \dots, n-1\}$, respectively.

For example, consider the following matrices, A and B :

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \end{pmatrix} \text{ and } B = \begin{pmatrix} a & b \\ c & d \\ e & f \\ g & h \\ i & j \end{pmatrix}$$

The bicyclic encoding of A is $\varphi(A) = (1, 7, 13, 4, 10, 11, 2, 8, 14, 5, 6, 12, 3, 9, 15)$ and the bicyclic encoding of B is $\varphi(B) = (a, d, e, h, i, b, c, f, g, j)$. Intuitively, the bicyclic encoding traverses the matrix along a diagonal, and when the traversal reaches the matrix boundary, it wraps around to the opposite side, continuing the diagonal pattern. This process ensures that every entry is eventually visited exactly once; for this to occur, the number of rows and columns must be coprime. The term *bicyclic encoding* is used because the encoding cycles through both dimensions, the rows and columns, of a matrix.

By the Chinese Remainder Theorem (CRT), the bicyclic encoding can be decoded with

$$\varphi^{-1}(\varphi(A)) = A$$

$$= [a_{i,j}] = \varphi(A)_{(i \cdot n \cdot (n^{-1} \bmod m) + j \cdot m \cdot (m^{-1} \bmod n)) \bmod (m \cdot n)}$$

where $(n^{-1} \bmod m)$ and $(m^{-1} \bmod n)$ are modular inverses of n modulo m and m modulo n , respectively. More information about CRT may be found in Appendix A.

4.2 Matrix Multiplications

Under the bicyclic encoding, multiplying two matrices is the sum of the pairwise multiplications of periodic rotations of their bicyclic encodings. This results in another bicyclically encoded matrix. Hence, these matrix multiplications take advantage of the homomorphic encryption operations of pairwise multiplication, pairwise addition, and rotation operations. As an example with the prior matrices A and B , we can calculate $A \cdot B$ as follows, where i and j are the rotational shifts required for each respective bicyclic encoding:

i	j	$\rho(\varphi(A), i)$	$\rho(\varphi(B), j)$	$\rho(\varphi(A), i) \odot \rho(\varphi(B), j)$
0	0	(1 7 13 4 10 11)	(a d e h i b)	(1a 7d 13e 4h 10i 11b)
6	6	(2 8 14 5 6 12)	(c f g j a d)	(2c 8f 14g 5j 6a 12d)
12	2	(3 9 15 1 7 13)	(e h i b c f)	(3e 9h 15i 1b 7c 13f)
3	8	(4 10 11 2 8 14)	(g j a d e h)	(4g 10j 11a 2d 8e 14h)
9	4	(5 6 12 3 9 15)	(i b c f g j)	(5i 6b 12c 3f 9g 15j)

Besides translating neatly into SIMD homomorphic encryption operations, matrix multiplications with bicyclic encodings offer several advantages:

- **Optimal Multiplicative Depth of 1:** This results in fewer bootstrapping operations and/or smaller cryptographic parameter sizes in a deep neural network.
- **Generality:** The algorithm generalizes to both ciphertext-plaintext and ciphertext-ciphertext matrix multiplications.
- **Consistent Input and Output Encodings:** Both the inputs and outputs are bicyclic encodings, which is useful for computations of successive matrix multiplications such as in deep neural networks because there is less need to change packings.
- **Free Transpose:** The bicyclic encoding calculates the transpose for free since a matrix and its transpose have the same bicyclic encoding [3]. Formally, for a matrix A , $\varphi(A) = \varphi(A^T)$.

However, these matrix multiplications under homomorphic encryption have a few limitations. For the following discussion, assume that an $(m \times n)$ matrix is multiplied by an $(n \times p)$ matrix:

- **Coprimeness:** All three of m , n , and p must be coprime to each other. Formally, $\gcd(m, n, p) = 1$. Practically, matrices can be padded with zeroes to coprime dimensions.
- **Slot Efficiency:** To satisfy correctness, the bicyclic encoding may need to be duplicated in the ciphertext, so only half of the available slots in a ciphertext may be used before duplication (with the rest padded with zeroes) in BMM-I. This means that for s available ciphertext slots, $mn < s/2$, $np < s/2$, and $mp < s/2$.
- **Expensive Rotations:** After multiplicative depth, rotation operations are the next performance bottleneck, constituting much of the computational costs of matrix multiplications. In BMM-I, $2n - 2$ rotations are required for ciphertext-ciphertext matrix multiplications and $n - 1$ rotations are required for ciphertext-plaintext matrix multiplications. (A few more rotations may be needed to duplicate the ciphertext, but these are much fewer.)

5 BATCH MATRIX MULTIPLICATION WITH TRICYCLIC ENCODINGS

Our first innovation is in proposing the novel *tricyclic encodings*, which encode three-dimensional tensors to enable batch matrix multiplications. Our motivation for this approach is that transformers have multiple self-attention heads. Can multiple, independent self-attention heads be computed in parallel using the SIMD properties of homomorphic encryption algorithms? We answer this question affirmatively by developing *batch matrix multiplication* algorithms in homomorphic encryption, allowing for fewer computational operations, fewer ciphertexts required to bootstrap, all while having the *optimal* multiplicative depth of 1. Our approach comes with virtually no additional cost, meaning that parallelizing h attention-heads will reduce the number of operations by nearly a factor of h .

5.1 Tricyclic Encoding

Let $m, n, p \in \mathbb{N}$ with $\gcd(m, n, p) = 1$. Consider the three-dimensional tensor

$$A = [a_{i,j,k}] \in \mathbb{R}^{m \times n \times p}.$$

A three-dimensional tensor is composed of m matrices with size $(n \times p)$. The *tricyclic encoding* of A is defined by the mapping

$$\varphi: \mathbb{R}^{m \times n \times p} \rightarrow \mathbb{R}^{m \cdot n \cdot p},$$

given by

$$\varphi(A)_k = a_{k \bmod m, k \bmod n, k \bmod p}, \quad \text{for all } k \in \mathbb{Z}_{mnp}.$$

The tricyclic encoding is a strict generalization of the bicyclic encoding since a matrix can be represented as a three-dimensional tensor with its first dimension $m = 1$. As Chen *et al.* notes, this can be generalized even further to an n -cyclic encoding which encodes for tensors [3].

5.1.1 Free Transpose. The tricyclic encoding retains the free transpose property of the bicyclic encoding. In fact, for any n -dimensional tensor, the n -cyclic encoding of the tensor and any of its transposes are equivalent. Hence, every transpose is effectively free. We prove this theorem in Appendix B. This free transpose property is useful for calculating K^T in self-attention.

5.2 Batch Matrix Multiplication

We develop three different matrix multiplication operations with tricyclic encodings that are each required for calculating multi-head self-attention:

- **Batch Matrix Multiplication:** Given an $(h \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , pairwise multiply the h distinct $(m \times n)$ matrices by h distinct $(n \times p)$ matrices to obtain h distinct $(m \times p)$ matrices, represented by an $(h \times m \times p)$ tensor. This has the effect of parallelizing multiple matrix multiplications and is primarily useful for calculating h different attention heads for $Q \cdot K^T$ and the result of the softmax multiplied by V .
- **Broadcasted Batch Matrix Multiplication:** Given a $(1 \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , multiply the one $(m \times n)$ matrix in A by h distinct $(n \times p)$ matrices in B to obtain h distinct $(m \times p)$ matrices, represented by an $(h \times m \times p)$ tensor. This has the effect of distributing the matrix in A across each of the h matrices in B . This is useful for calculating h different attention heads for the query, key, and value matrices.
- **Concatenated Matrix Multiplication:** Given an $(h \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , pairwise multiply the h distinct $(m \times n)$ matrices by h distinct $(n \times p)$ matrices to obtain h distinct $(m \times p)$ matrices. Then, sum all h matrices to obtain one $(m \times p)$ matrix, which is represented by a $(1 \times m \times p)$ tensor.³ This is useful for calculating the last self-attention linear layer where h different attention heads are concatenated together before multiplying by a matrix.

We now derive each of these matrix multiplications.

5.2.1 Batch Matrix Multiplication. For an $(h \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , the batch matrix multiplication is expressed as

$$Z_{i,j,k} = \left[\sum_{c=0}^{n-1} a_{i,j,c} \cdot b_{i,c,k} \right] \in \mathbb{R}^{h \times m \times p}. \quad (4)$$

³This can also be viewed as the following: Concatenate the h distinct matrices in A column-wise to obtain a $(m \times h \cdot n)$ matrix. Also, concatenate the h distinct matrices in B row-wise to obtain a $(h \cdot n \times p)$ matrix. Finally, multiply the two matrices together to obtain a $(m \times p)$ matrix.

$A \in \mathbb{R}^{2 \times 5 \times 7} = [a_{i,j,k}]$ where $i \in \mathbb{Z}_2, j \in \mathbb{Z}_5, k \in \mathbb{Z}_7$ and $B \in \mathbb{R}^{2 \times 7 \times 3} = [b_{i,j,k}]$ where $i \in \mathbb{Z}_2, j \in \mathbb{Z}_7, k \in \mathbb{Z}_3$

$$\varphi(A) = \begin{pmatrix} a_{0,0,0} & a_{1,1,1} & a_{0,2,2} & a_{1,3,3} & a_{0,4,4} & a_{1,0,5} & a_{0,1,6} & a_{1,2,0} & a_{0,3,1} & a_{1,4,2} & a_{0,0,3} & a_{1,1,4} & a_{0,2,5} & a_{1,3,6} & \cdots \end{pmatrix}$$

$$\varphi(B) = \begin{pmatrix} b_{0,0,0} & b_{1,1,1} & b_{0,2,2} & b_{1,3,0} & b_{0,4,1} & b_{1,5,2} & b_{0,6,0} & b_{1,0,1} & b_{0,1,2} & b_{1,2,0} & b_{0,3,1} & b_{1,4,2} & b_{0,5,0} & b_{1,6,1} & \cdots \end{pmatrix}$$

i	j	$\rho(\varphi(A), i)$	$\rho(\varphi(B), j)$	$\rho(\varphi(A), i) \odot \rho(\varphi(B), j)$
0	0	$(a_{0,0,0} \ a_{1,1,1} \ a_{0,2,2} \ a_{1,3,3} \ a_{0,4,4} \ \cdots)$	$(b_{0,0,0} \ b_{1,1,1} \ b_{0,2,2} \ b_{1,3,0} \ b_{0,4,1} \ \cdots)$	$(a_{0,0,0} \cdot b_{0,0,0} \ a_{1,1,1} \cdot b_{1,1,1} \ a_{0,2,2} \cdot b_{0,2,2} \ a_{1,3,3} \cdot b_{1,3,0} \ a_{0,4,4} \cdot b_{0,4,1} \ \cdots)$
50	36	$(a_{0,0,1} \ a_{1,1,2} \ a_{0,1,3} \ a_{1,3,4} \ a_{0,4,5} \ \cdots)$	$(b_{0,1,0} \ b_{1,2,1} \ b_{0,3,2} \ b_{1,4,0} \ b_{0,5,1} \ \cdots)$	$(a_{0,0,1} \cdot b_{0,1,0} \ a_{1,1,2} \cdot b_{1,2,1} \ a_{0,1,3} \cdot b_{0,3,2} \ a_{1,3,4} \cdot b_{1,4,0} \ a_{0,4,5} \cdot b_{0,5,1} \ \cdots)$
30	30	$(a_{0,0,2} \ a_{1,1,3} \ a_{0,1,4} \ a_{1,3,5} \ a_{0,4,6} \ \cdots)$	$(b_{0,2,0} \ b_{1,3,1} \ b_{0,4,2} \ b_{1,5,0} \ b_{0,6,1} \ \cdots)$	$(a_{0,0,2} \cdot b_{0,2,0} \ a_{1,1,3} \cdot b_{1,3,1} \ a_{0,1,4} \cdot b_{0,4,2} \ a_{1,3,5} \cdot b_{1,5,0} \ a_{0,4,6} \cdot b_{0,6,1} \ \cdots)$
10	24	$(a_{0,0,3} \ a_{1,1,4} \ a_{0,1,5} \ a_{1,3,6} \ a_{0,4,0} \ \cdots)$	$(b_{0,3,0} \ b_{1,4,1} \ b_{0,5,2} \ b_{1,6,0} \ b_{0,0,1} \ \cdots)$	$(a_{0,0,3} \cdot b_{0,3,0} \ a_{1,1,4} \cdot b_{1,4,1} \ a_{0,1,5} \cdot b_{0,5,2} \ a_{1,3,6} \cdot b_{1,6,0} \ a_{0,4,0} \cdot b_{0,0,1} \ \cdots)$
60	18	$(a_{0,0,4} \ a_{1,1,5} \ a_{0,1,6} \ a_{1,3,0} \ a_{0,4,1} \ \cdots)$	$(b_{0,4,0} \ b_{1,5,1} \ b_{0,6,2} \ b_{1,0,0} \ b_{0,1,1} \ \cdots)$	$(a_{0,0,4} \cdot b_{0,4,0} \ a_{1,1,5} \cdot b_{1,5,1} \ a_{0,1,6} \cdot b_{0,6,2} \ a_{1,3,0} \cdot b_{1,0,0} \ a_{0,4,1} \cdot b_{0,1,1} \ \cdots)$
40	12	$(a_{0,0,5} \ a_{1,1,6} \ a_{0,1,0} \ a_{1,3,1} \ a_{0,4,2} \ \cdots)$	$(b_{0,5,0} \ b_{1,6,1} \ b_{0,0,2} \ b_{1,1,0} \ b_{0,2,1} \ \cdots)$	$(a_{0,0,5} \cdot b_{0,5,0} \ a_{1,1,6} \cdot b_{1,6,1} \ a_{0,1,0} \cdot b_{0,0,2} \ a_{1,3,1} \cdot b_{1,1,0} \ a_{0,4,2} \cdot b_{0,2,1} \ \cdots)$
20	6	$(a_{0,0,6} \ a_{1,1,0} \ a_{0,1,1} \ a_{1,3,2} \ a_{0,4,3} \ \cdots)$	$(b_{0,6,0} \ b_{1,0,1} \ b_{0,1,2} \ b_{1,2,0} \ b_{0,3,1} \ \cdots)$	$(a_{0,0,6} \cdot b_{0,6,0} \ a_{1,1,0} \cdot b_{1,0,1} \ a_{0,1,1} \cdot b_{0,1,2} \ a_{1,3,2} \cdot b_{1,2,0} \ a_{0,4,3} \cdot b_{0,3,1} \ \cdots)$

Figure 2: Batch Matrix Multiplication with Tricyclic Encoding. The first two columns of the table list the rotational shifts i and j required for the tricyclic encodings $\varphi(A)$ and $\varphi(B)$, respectively. The third and fourth columns display the corresponding rotations of these encodings. The rotational shifts for $\varphi(A)$ occur every 50 mod 70, derived by setting $h = 2, m = 5$, and $n = 7$ in the expression $(chm((hm)^{-1} \bmod n)) \bmod (hmn)$ for $c = 0, 1, \dots, n-1$. Likewise, the rotational shifts for $\varphi(B)$ occur every 36 mod 42, obtained by substituting $h = 2, n = 7$, and $p = 3$ into $(chp((hp)^{-1} \bmod n)) \bmod (hnp)$. The final column contains the element-wise product of the third and fourth columns, and summing these values pairwise yields the tricyclic encoding $\varphi(A \cdot B)$.

Applying the definition of tricyclic encoding to Z , we get

$$\varphi(Z)_k = \sum_{c=0}^{n-1} \left(a_{k \bmod h, k \bmod m, (k+c) \bmod n} \cdot b_{k \bmod h, (k+c) \bmod n, k \bmod p} \right). \quad (5)$$

Using CRT, we can determine that $x = k + chm((hm)^{-1} \bmod n) \bmod (hmn)$ is a solution for the congruence $x \equiv k \bmod h, x \equiv k \bmod m$, and $x \equiv k + c \bmod n$ (see Appendix A). Using this solution for x helps us obtain the following:

$$= \sum_{c=0}^{n-1} \left(a_{x \bmod h, x \bmod m, x \bmod n} \cdot b_{k \bmod h, (k+c) \bmod n, k \bmod p} \right). \quad (6)$$

Note that Equation 6 simplifies to Equation 5, proving that the two are equivalent. Next, we can apply the definition of the tricyclic encoding to A and obtain

$$= \sum_{c=0}^{n-1} \left(\varphi(A)_{(k+chm((hm)^{-1} \bmod n)) \bmod (hmn)} \odot b_{k \bmod h, (k+c) \bmod n, k \bmod p} \right). \quad (7)$$

Likewise, we can apply the same steps to B and get

$$= \sum_{c=0}^{n-1} \left(\varphi(A)_{(k+chm((hm)^{-1} \bmod n)) \bmod (hmn)} \odot \varphi(B)_{(k+chp((hp)^{-1} \bmod n)) \bmod (hnp)} \right). \quad (8)$$

Finally, assuming sufficient duplication of $\varphi(A)$ and $\varphi(B)$ and rotational correctness, we can simplify to

$$\varphi(Z) = \sum_{c=0}^{n-1} \left(\rho \left(\varphi(A), chm((hm)^{-1} \bmod n) \bmod (hmn) \right) \odot \rho \left(\varphi(B), chp((hp)^{-1} \bmod n) \bmod (hnp) \right) \right). \quad (9)$$

This shows that we can calculate the batch matrix multiplication by computing rotations of every $hm((hm)^{-1} \bmod n)$ and $hp((hp)^{-1} \bmod n)$ for $\varphi(A)$ and $\varphi(B)$, respectively, and summing their product. In Figure 2, we show an example of a batch matrix multiplication for two tensors A and B with tricyclic encodings. Lastly, note that when $h = 1$, the equation simplifies to the same as a matrix multiplication, showing that batch matrix multiplications with tricyclic encodings are a strict generalization of bicyclic matrix multiplications.

5.2.2 Broadcasted Batch Matrix Multiplication. For a $(1 \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , the broadcasted batch matrix multiplication is expressed as

$$Z_{i,j,k} = \left[\sum_{c=0}^{n-1} a_{0,j,c} \cdot b_{i,c,k} \right] \in \mathbb{R}^{h \times m \times p}. \quad (10)$$

We prove in Appendix C.1 that the broadcasted batch matrix multiplication is achieved by the following operations:

$$\varphi(Z) = \sum_{c=0}^{n-1} \left(\rho \left(\varphi(A), cm(m^{-1} \bmod n) \bmod (mn) \right) \odot \rho \left(\varphi(B), chp((hp)^{-1} \bmod n) \bmod (hnp) \right) \right). \quad (11)$$

This means that we can compute the broadcasted batch matrix multiplication by computing rotations of every $m(m^{-1} \bmod n)$ and $hp((hp)^{-1} \bmod n)$ for $\varphi(A)$ and $\varphi(B)$, respectively, and summing their product.

5.2.3 Concatenated Matrix Multiplication. For an $(h \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , the concatenated matrix multiplication

is expressed as

$$Z_{0,j,k} = \left[\sum_{d=0}^{h-1} \sum_{c=0}^{n-1} a_{d,j,c} \cdot b_{d,c,k} \right] \in \mathbb{R}^{1 \times m \times p}. \quad (12)$$

We prove in Appendix C.2 that this can be computed using tricyclic encodings as follows:

$$\begin{aligned} \varphi(Z) &= \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} (\rho(\varphi(A), (d \cdot r_1 + c \cdot r_2) \bmod (h \cdot m \cdot n)) \odot \\ &\quad \rho(\varphi(B), (d \cdot r_3 + c \cdot r_4) \bmod (h \cdot n \cdot p))) \\ &\quad \text{where } r_1 = m \cdot n \cdot ((m \cdot n)^{-1} \bmod h), \\ &\quad r_2 = h \cdot m \cdot ((h \cdot m)^{-1} \bmod n), \\ &\quad r_3 = n \cdot p \cdot ((n \cdot p)^{-1} \bmod h), \\ &\quad r_4 = h \cdot p \cdot ((h \cdot p)^{-1} \bmod n). \end{aligned} \quad (13)$$

We briefly elucidate on Equation (13). The variables d and c are summation indices where d iterates over the different matrices in the tensorized operands A and B and c iterates over the columns of each matrix in A and the corresponding rows in each matrix in B . The values r_1 , r_2 , r_3 , and r_4 represent rotation strides. For $\varphi(A)$, r_1 is the rotation step needed to move from one matrix to the next while keeping the same row and column indices and r_2 is the rotation step needed to move from one column to the next within the same matrix and row. For $\varphi(B)$, r_3 is the rotation step needed to move from one matrix to the next while keeping the same row and column indices and r_4 is the rotation step needed to move from one row to the next within the same matrix and column.

6 OPTIMIZED TRICYCLIC MATMULS

Our next contribution improves both ciphertext-ciphertext and ciphertext-plaintext matrix multiplication under tricyclic encodings. For both cases, we apply the *Baby Step-Giant Step* (BSGS) algorithm [5] to reduce the number of rotation operations, significantly enhancing performance. In addition, our ciphertext-plaintext batch multiplication algorithms eliminates several constraints present in prior work [3], particularly those related to coprimeness and matrix size alignment. These improvements yield *strictly superior* ciphertext-ciphertext and ciphertext-plaintext multiplication algorithms compared to those based on bicyclic encodings [3].

6.1 Optimized Ciphertext-Plaintext Matmuls

In a transformer block, ciphertext-plaintext matrix multiplications are used for every matrix multiplication involving multiplying by learned model weights and are the majority of matrix multiplications. Hence, it's critical to ensure that operations are easily applicable (i.e. fewer limitations) and efficient. Consider the batch matrix multiplication between a ciphertext tensor of shape $(h_1 \times m \times n)$ and a plaintext tensor of shape $(h_2 \times n \times p)$, yielding a ciphertext tensor of shape $(h_3 \times m \times p)$. Specific cases of the number of matrices h_1 , h_2 , and h_3 in the ciphertext input tensor, plaintext input tensor, and the output tensor, respectively, are the following matrix multiplications that we have discussed:

- Matrix Multiplication: $h_1 = h_2 = h_3 = 1$
- Batch Matrix Multiplication: $h_1 = h_2 = h_3$

- Broadcasted Batch Matrix Multiplication: $h_1 = 1, h_2 = h_3$
- Concatenated Matrix Multiplication: $h_1 = h_2, h_3 = 1$

Our enhancements to ciphertext-plaintext matrix multiplications generalize to all four matrix multiplications above and improve over prior work [3] in several ways:

- **Coprimeness:** Only the input and output ciphertext matrices need to satisfy coprime dimensions. The plaintext matrix need not satisfy coprime dimensions, meaning that only $\gcd(h_1, m, n) = \gcd(h_3, m, p) = 1$.
- **Larger Plaintext Matrix Size:** The plaintext matrix need not fit into a fixed number of slots in a ciphertext. In fact, the plaintext matrix can have more total values than slots.
- **Slot Efficiency:** Instead of the ciphertext matrices only being able to use up to half the available slots, the only requirement is that the number of encrypted values fits into the number of available slots. Formally, for s slots, the input ciphertext matrix size $h_1 m n < s$ and output ciphertext matrix size $h_3 m p < s$.
- **Fewer Rotations:** We minimize the number of rotations required by using the Baby-Step Giant-Step (BSGS) algorithm [5]. Without accounting for duplication, this reduces the number of rotations from $n - 1$ to between $2\sqrt{n} - 2$ and $2\sqrt{2n} - 2$ depending on the output matrix size. For the concatenated matrix multiplication, it reduces the number of rotations from $h_1 n - 1$ to between $2\sqrt{h_1 n} - 2$ and $2\sqrt{2h_1 n} - 2$.

As an illustrative example that demonstrates our approach and its fewer limitations, suppose that there are $s = 8$ available slots in a ciphertext. Consider the $(1 \times 2 \times 3)$ ciphertext tensor A and the $(1 \times 3 \times 3)$ plaintext tensor B :

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \text{ and } B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The tricyclic encoding of A , $\varphi(A)$ is (a, e, c, d, b, f) and is encrypted with zero padding for the remaining 2 slots. Then the batch matrix multiplication can be computed by multiplying rotations of the ciphertext and diagonals of the plaintext as follows:

i	$\rho(\varphi(A), i)$	BatchDiag(B)	$\rho(\varphi(A), i) \odot \text{BatchDiag}(B)$
0	$(a \ e \ c \ d \ b \ f \ 0 \ 0)$	$(1 \ 5 \ 9 \ 1 \ 5 \ 9 \ 0 \ 0)$	$(1a \ 5e \ 9c \ 1d \ 5b \ 9f \ 0 \ 0)$
4	$(b \ f \ 0 \ 0 \ a \ e \ c \ d)$	$(4 \ 8 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$	$(4b \ 8f \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$
-2	$(0 \ 0 \ a \ e \ c \ d \ b \ f)$	$(0 \ 0 \ 3 \ 4 \ 8 \ 3 \ 0 \ 0)$	$(0 \ 0 \ 3a \ 4e \ 8c \ 3d \ 0 \ 0)$
2	$(c \ d \ b \ f \ 0 \ 0 \ a \ e)$	$(7 \ 2 \ 6 \ 7 \ 0 \ 0 \ 0 \ 0)$	$(7c \ 2d \ 6b \ 7f \ 0 \ 0 \ 0 \ 0)$
-4	$(b \ f \ 0 \ 0 \ a \ e \ c \ d)$	$(0 \ 0 \ 0 \ 0 \ 2 \ 6 \ 0 \ 0)$	$(0 \ 0 \ 0 \ 0 \ 2a \ 6e \ 0 \ 0)$

By summing the right column, we obtain $\varphi(A \cdot B)$ with zero padding for extra slots. Multiplying by a plaintext is inherently more flexible than multiplying by a ciphertext since plaintext values can be rearranged at no cost and do not need to be represented with a tricyclic encoding to ensure correctness. Our method for determining the plaintext diagonal is similar to how tricyclic encoding traverses the tensor, except that it does not require every value in the tensor to be visited exactly once. Hence, there is no coprimeness or size requirement for the plaintext tensor.

We derive the diagonal traversal method for batch matrix multiplication and defer the remaining matrix multiplication algorithms

to the Appendix. Recall that the batch matrix multiplication $Z = A \cdot B$ from Equation (4) allows us to simplify to Equation (7) as follows:

$$\varphi(Z)_k = \sum_{c=0}^{n-1} \left(\varphi(A)_{(k+chm((hm)^{-1} \bmod n)) \bmod (hmn)} \odot b_{k \bmod h, (k+c) \bmod n, k \bmod p} \right) \quad (14)$$

Assuming sufficient duplication of $\varphi(A)$ and rotational correctness, we can simplify to

$$\varphi(Z) = \sum_{c=0}^{n-1} (\rho(\varphi(A), (chm((hm)^{-1} \bmod n)) \bmod (hmn)) \odot \text{BatchDiag}(B, c)) \quad (15)$$

where $\text{BatchDiag}(B, c)_k = b_{k \bmod h, (k+c) \bmod n, k \bmod p}$ for $k \in \mathbb{Z}_{hmp}$. Note that Equation (15) does not assume a tricyclic encoding for B , avoiding the coprimeness requirement.

Rotational correctness is not always guaranteed such as in the example of $A \cdot B$ above, where rotating $\varphi(A)$ by 4 needs to contain (b, f, a, e, c, d) for the correctness of matrix multiplication. We address this by modifying $\text{BatchDiag}(B, c)$ to be a mask for the correct number of rotational slots, which in this case masks out everything but the first two correct slots. To fix the remaining incorrect slots, we perform additional rotations (in this case, a -2 rotation) to obtain the remaining correct slots (the latter 4 values), which are then also masked by a plaintext properly. Masking with the plaintext to fix rotational correctness is the main reason ciphertext tensors are not required to leave half the slots available for duplication. Our algorithm can still take advantage of duplicating the matrix in the ciphertext for rotational correctness provided there are enough free slots. This can remove some or all of the additional rotations required for correctness.

6.2 Baby-Step Giant-Step Algorithm

We reduce the number of rotations required in our matrix multiplications by using the Baby-Step Giant-Step (BSGS) algorithm [5]. The BSGS algorithm for optimizing rotations can be applied whenever there is a summation of several periodically incrementing rotations. This is especially convenient for matrix multiplications with tricyclic and bicyclic encodings because the matrix multiplications take advantage of the inherent dimensional periodicity of these encoding schemes.

6.2.1 Ciphertext-Plaintext Matmuls. We now describe the BSGS algorithm and its application to our ciphertext-plaintext matrix multiplications. Let T be the periodicity of rotations and N be the number of sums of rotations and multiplications required. Let the number of baby steps be $N_1 = O(\sqrt{N})$ that we can precompute. For convenience, let N_1 divide N evenly and the number of giant steps be $N_2 = N/N_1$. Let x be a ciphertext and y be a plaintext.

$$\sum_{c=0}^{N-1} \rho(x, T \cdot c) \odot y = \sum_{j=0}^{N_2-1} \sum_{i=0}^{N_1-1} \rho(x, T \cdot (i + j \cdot N_1)) \odot y \quad (16)$$

$$= \sum_{j=0}^{N_2-1} \rho \left(\sum_{i=0}^{N_1-1} \rho(x, T \cdot i) \odot \rho(y, -T \cdot N_1 \cdot j), T \cdot N_1 \cdot j \right). \quad (17)$$

We remark that applying BSGS does not require $N_1 \cdot N_2 = N$ exactly, and only requires that every rotation be grouped into sets of giant steps, where the differences between rotations sizes align with precomputed baby steps. Rotating the plaintext, $\rho(y, -T \cdot N_1 \cdot j)$, is effectively at no cost, while the baby step rotations $\rho(x, T \cdot i)$ for $0 \leq i < N_1$ can be precomputed once. By choosing appropriate N_1 and N_2 , and substituting for the periodicity $T = hm$ we can reduce the number of rotations in our ciphertext-plaintext matrix multiplications to between $2\sqrt{n} - 2$ and $2\sqrt{2n} - 2$.

Appendix D shows that Equation (15) simplifies to

$$\sum_{c=0}^{n-1} (\rho(\varphi(A), c \cdot hm) \odot \text{BatchDiag}'(B, c)) \quad (18)$$

where $\text{BatchDiag}'(B, c)_k = b_{k \bmod h, (k+c \cdot hm) \bmod n, k \bmod p}$ for $k \in \mathbb{Z}_{hmp}$. This proves that BSGS can be applied for $x = \varphi(A)$, $y = \text{BatchDiag}'(B, c)$, and $T = hm$.

6.2.2 Ciphertext-Ciphertext Matmuls. Our next innovation is in applying BSGS to ciphertext-ciphertext matrix multiplications. We observe that these matrix multiplications can be expressed in the following form, where x and y are both ciphertexts, T is the periodicity of rotations of x , and $\text{RotY}(c)$ represents the rotations required for y in the c -th summation term.

$$\begin{aligned} & \sum_{c=0}^{N-1} \rho(x, T \cdot c) \odot \rho(y, \text{RotY}(c)) \quad (19) \\ &= \sum_{j=0}^{N_2-1} \sum_{i=0}^{N_1-1} \rho(x, T(i + j \cdot N_1)) \odot \rho(y, \text{RotY}(i + j \cdot N_1)) \quad (20) \\ &= \sum_{j=0}^{N_2-1} \rho \left(\sum_{i=0}^{N_1-1} \rho(x, T \cdot i) \odot \rho(y, \text{RotY}(i + j \cdot N_1) - T \cdot N_1 \cdot j), T \cdot N_1 \cdot j \right). \quad (21) \end{aligned}$$

BSGS decreases the number of rotations required for ciphertext-ciphertext matrix multiplications from $2n - 2$ rotations to only around $n + 2\sqrt{n} - 3$ rotations. This is because although y is still rotated $n - 1$ times, the number of rotations of x decreases to $N_1 - 1 \approx \sqrt{x} - 1$ for the number of precomputed baby steps of x and $N_2 - 1 \approx \sqrt{x} - 1$ giant steps to rotate the product of rotations of x and y . We prove in Appendix E that batch matrix multiplications under tricyclic encodings follow the form

$$\varphi(Z) = \sum_{c'=0}^{n-1} (\rho(\varphi(A), (c' \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), (hp(c' \cdot m(p)^{-1} \bmod n)) \bmod (hnp))) \quad (22)$$

proving that BSGS can be applied for $x = \varphi(A)$, $y = \varphi(B)$, periodicity $T = hm$, and $\text{RotY}(c) = (hp(c \cdot m(p)^{-1} \bmod n)) \bmod (hnp)$.

7 NONLINEAR LAYERS

We approximate the nonlinear layers of the transformer using polynomials. Among these, only softmax and layer normalization require special attention to packing, as they involve row-wise sums prior to computing an inverse or inverse square root. For a tricyclically encoded $(h \times m \times n)$ tensor, summing up values in the same

Table 2: Comparison of Softmax Approaches

Work	Stable	Faithful Approx.	Mult. Depth
Rovida <i>et al.</i> [25]	✗	✓	15
NEXUS [26]	✗	✓	16
Rho <i>et al.</i> [24]	✓	✗	16
Powerformer [23]	✓	✗	6
Castro <i>et al.</i> [9]	✓	✓	133
Cho <i>et al.</i> [8]	✓	✓	40
Thor [21]	✓	✓	30
Tricycle (ours)	✓	✓	17

matrix and row only requires rotation increments of $h \cdot m$ since values in the same matrix and row are every $h \cdot m$ values in the tricyclic encoding. Our algorithms for evaluating row-wise summations require $O(\log n)$ rotations and are described in Appendix F, demonstrating that tricyclic encodings efficiently support row-wise operations.

7.1 Softmax

To calculate attention, a softmax calculation is required. Softmax is defined as

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}. \quad (23)$$

In plaintext settings, softmax can suffer from numerical instability because exponentiating large input values may lead to overflow, while very negative values can yield outputs so small they become indistinguishable from zero. A common stabilization technique is to subtract the maximum input value from all terms, taking advantage of the fact that softmax is invariant to adding or subtracting the same constant across all inputs. This ensures that the largest input becomes zero so exponentiating it yields 1, and the remaining values lie between 0 and 1, greatly improving numerical stability. Under homomorphic encryption, however, these issues are exacerbated. Large values accelerate ciphertext noise growth, while small values are more easily lost to noise. Applying the same stabilization technique is far more difficult in this setting because computing the maximum value under encryption is both nontrivial and computationally expensive.

Table 2 compares the different softmax approaches under homomorphic encryption in the literature. The table reveals a trilemma in the existing literature: no prior method simultaneously achieves all three of stability, faithful approximation of softmax, and low multiplicative depth. A *faithful approximation* of softmax closely replicates its behavior, enabling it to serve as a drop-in replacement without requiring model retraining. Powerformer [23] and Rho *et al.* [24], for instance, do not faithfully approximate softmax, instead opting for more homomorphic encryption-friendly alternatives that prioritize stability. However, these alternatives typically require retraining and can lead to degraded model accuracy due to their departure from the original softmax behavior. Other methods preserve the approximation and maintain low multiplicative depth but give up on stability. NEXUS [26], for example, attempts to stabilize outputs by subtracting a fixed constant from all inputs. This heuristic falls short because (i) if the constant is too small,

exponentiation results in large numbers and (ii) if the constant is too large, exponentiation vanishes to zero. Lastly, some methods provide both stability and faithful approximation, but at the cost of high multiplicative depth. Castro *et al.* [9] achieve this by computing the maximum under homomorphic encryption, incurring a huge multiplicative depth of 133. Thor [21] and Cho *et al.* [8] apply iterative squaring and normalization to stabilize softmax but these methods also require a multiplicative depth of at least 30.

Statistical Max Estimation: We resolve the trilemma of stability, faithful approximation, and low multiplicative depth of 17 by presenting a straightforward yet effective method to stabilize softmax. Our approach is based on the following question: *Could the maximum be estimated for stability from statistics such as mean and variance that are easy to calculate under homomorphic encryption?*

To answer this question, we leverage a result from extreme value theory. Suppose there are n independent and identically distributed random variables $X_1, X_2, \dots, X_n$ drawn from the Gaussian distribution $X_i \sim \mathcal{N}(\mu, \sigma^2)$. Define $Y = \max_{1 \leq i \leq n} X_i$. It is known that that the expected maximum can be upper bounded [17] as follows:

$$\mathbb{E}[Y] \leq \mu + \sigma \sqrt{2 \ln(n)}. \quad (24)$$

Assuming that the inputs to the softmax function are approximately Gaussian, we first compute their mean and variance under homomorphic encryption, and then apply this inequality to estimate the maximum. To reduce the computational cost and multiplicative depth of calculating the standard deviation, σ , we approximate the square root with a simple linear expression ($\sigma \approx \alpha_0 + \alpha_1 \cdot \sigma^2$). We manually tune $\alpha_0 = 0.1$ and $\alpha_1 = 1.15$ to reasonably approximate $\sqrt{x}$ up to 100. Because we only require a rough estimate rather than the exact value, this approximation is adequate for our purposes.

To estimate softmax itself, we estimate the exponentiation with

$$e^x \approx \left(1 + \frac{x}{2^k}\right)^{2^k} \quad (25)$$

for $k = 7$. We aggressively fold the $1/2^k$ constant used for exponentiation with the max estimation subtraction, which results in the max estimation costing only one extra multiplicative depth, which is inexpensive. Next, we approximate the inverse with the Chebyshev approximation [4] in the range $[0.1, n]$ for n input values and one round of Goldschmidt’s Iteration [11] to improve the approximation. A primary benefit of overestimating the maximum in Equation (24) is that we can bound the maximum sum of exponentiations to be less than n since all exponentiations will be less than 1. In total, we use 9 multiplicative depth levels for the statistical max estimation and exponentiation and 8 multiplicative depth levels for the inverse.

7.2 Layer Normalization

Each transformer block applies layer normalization twice: once after the attention layer’s residual connection, and again after the feedforward layer’s residual connection. In both cases, the normalization is performed row-wise over matrices. For a given row vector $x \in \mathbb{R}_n$ with normalization weights $\gamma \in \mathbb{R}_n$ and biases $\beta \in \mathbb{R}_n$ the layer normalization is defined as

$$\text{LayerNorm}(x)_i = \frac{x_i - \mu}{\sigma} \cdot \gamma_i + \beta_i \quad (26)$$

where μ is the mean of x and σ is its standard deviation.

To compute this normalization, we first calculate the mean and variance in the standard way. Finally, we approximate the inverse square root of the variance using a Chebyshev polynomial approximation [4].

7.3 GELU & Tanh

Every transformer block employs a GELU activation in its feed-forward network, while the output layers of the transformer model use the hyperbolic tangent (tanh) function. GELU is defined as

$$\text{GELU}(x) = \frac{x}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right] \quad (27)$$

where the error function $\text{erf}(x)$ is given as

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \quad (28)$$

Meanwhile, tanh is defined as

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (29)$$

To evaluate both GELU and tanh efficiently, we use Chebyshev approximations [4], similar to the system by Roveda *et al.* [25].

8 TRICYCLE PROTOCOL

We now present the end-to-end Tricycle system, which provides private inference on transformer models. Transformer inference relies heavily on matrix multiplications, which are parameterized by the model’s hidden dimension d_{model} , the number of attention heads h , the attention head dimension $d_k = d_{\text{model}}/h$, and the feedforward network size d_{ff} , typically equal to $4 \cdot d_{\text{model}}$. To enable the efficient evaluation of transformers under homomorphic encryption, Tricycle utilizes batch matrix multiplications under tricyclic encodings. To support tricyclic encodings, it pads key dimensions such as d_{model} and d_k to prime numbers, ensuring coprimeness. Additionally, when tensors are too large to fit within a single ciphertext, they are divided into smaller tensor blocks, enabling block-wise batch matrix multiplications [3]. For example, the feedforward layers may be represented as multiple matrix blocks to support computation within ciphertext size constraints. In the following discussion, we assume that all dimensions have been padded for coprimeness and that large ciphertext matrices have been appropriately partitioned for block-wise matrix multiplication and will refer to the transformer architecture illustrated in Figure 1 using the same notation introduced in Section 3.1.

- (1) **Token Padding:** To enable private inference on a sequence of N tokens, the client first appends padding tokens, extending the input to a total of L tokens. The padding ensures that the dimensions used in subsequent matrix multiplications are coprime. Since LLMs are designed to treat padding tokens as inert, they do not influence the final inference result.
- (2) **Embedding Layer:** The client computes the embedding layer locally in plaintext, producing an $(L \times d_{\text{model}})$ matrix that represents the embedded token sequence.
- (3) **Input to Transformer Blocks:** The embedded token matrix is tricyclically encoded as a $(1 \times L \times d_{\text{model}})$ tensor X ,

then encrypted and sent to the untrusted server. This ciphertext serves as the input to the first transformer block. For subsequent blocks, the server uses the ciphertext output of each block, which is also a $(1 \times L \times d_{\text{model}})$ tricyclic encoding, as input to the next block.

- (4) **Q in Attention:** The server can now homomorphically evaluate the query matrix $Q = X \cdot W_Q$ (a) using our broadcasted batch matrix multiplication algorithm between X and the weight matrix W_Q , represented by an $(h \times d_{\text{model}} \times d_k)$ tensor, to obtain an $(h \times L \times d_k)$ tensor.
- (5) **Reshape X :** Before calculating K and V , the server reshapes X to X' , resulting in a $(1 \times (L+r) \times d_{\text{model}})$ tensor where r is typically a small number such as 1 or 2. This is accomplished by multiplying $X' = I' \cdot X$ where I' is a $(1 \times (L+r) \times L)$ tensor, which is essentially the tall identity matrix where the top $L \times L$ matrix is the identity and the bottom r rows are zeroes. This operation is the only packing change in our entire protocol, costing only one multiplicative depth, and ensures coprimeness when computing $Q \cdot K^T$. An appropriate choice of r ensures that $L+r$ is coprime with h , which is needed to evaluate K and V .
- (6) **K and V in Attention:** The server evaluates K (b) and V (c) using a broadcasted batch matrix multiplication with $K = X' \cdot W_K$ and $V = X' \cdot W_V$. Weight matrices W_K and W_V are also represented by $(h \times d_{\text{model}} \times d_k)$ tensors and the resulting K and V are $(h \times (L+r) \times d_k)$ tensors.⁴
- (7) **Attention Score:** The server can now evaluate the attention scores $Q \cdot K^T / \sqrt{d_k}$ (e). Since transpose is provided for free by the tricyclic encoding, K^T is derived by treating K as a $(h \times d_k \times (L+r))$ tensor. To evaluate $Q \cdot K^T$, we use a modified ciphertext-ciphertext batch matrix multiplication to obtain a $(h \times L \times (L+r))$ tensor. Ciphertext-ciphertext matrix multiplications are not guaranteed to result in zero padding, which softmax requires for rotational correctness. Hence, Tricycle multiplies rotations of Q with a mask containing $1/\sqrt{d_k}$ in the first $h \cdot L \cdot (L+r)$ slots and zeroes elsewhere before multiplying that result by K^T to guarantee zero padding. This masking operation doesn’t incur any extra multiplicative depth because K^T already has one higher multiplicative depth than Q .
- (8) **Softmax:** The server calculates the row-wise softmax (f) over the $(h \times L \times (L+r))$ tricyclically encoded tensor.
- (9) **Attention Output:** The server calculates the attention output (g) by using our ciphertext-ciphertext batch matrix multiplication to multiply the softmax result with V , an $(h \times (L+r) \times d_k)$ tensor, to obtain an $(h \times L \times d_k)$ tensor.
- (10) **Attention Output Projection:** The output projection of multi-head attention (h) (i) is calculated by using our concatenated matrix multiplication algorithm, multiplying the attention output tensor by an output projection matrix W_O

⁴Appendix G explains how the reshape operation can be folded into the computation of K and V , eliminating the extra multiplicative depth reshaping would introduce. However, this comes at the cost of increased rotations and homomorphic multiplications and ultimately provides no net depth savings, since an extra multiplicative depth is still required later to enforce zero padding for softmax.

represented by an $(h \times d_k \times d_{\text{model}})$ tensor to obtain a $(1 \times L \times d_{\text{model}})$ tensor.

- (11) **First Residual Connection:** The server calculates the residual connection (j) by summing the output projection by X . Both the output projection ciphertext and input ciphertext X are tricyclic encodings of $(1 \times L \times d_{\text{model}})$ tensors and so can be added.
- (12) **First Layer Normalization:** The server calculates a row-wise layer normalization (k) . The output of the first layer normalization, a $(1 \times L \times d_{\text{model}})$ tensor is used as input to the first feedforward layer and second residual connection.
- (13) **First Feedforward Layer:** The server calculates the first feedforward layer (l) by multiplying the output of the layer normalization with the first feedforward matrix W_{ff_1} represented by a $(1 \times d_{\text{model}} \times d_{ff})$ tensor to obtain a ciphertext that encrypts a $(1 \times L \times d_{ff})$ tensor.
- (14) **GELU:** The server applies a Chebyshev approximation of the non-linear GELU function over the ciphertexts (m) .
- (15) **Second Feedforward Layer:** The server calculates the second feedforward layer (n) by multiplying the output of GELU with the second feedforward matrix W_{ff_2} represented by a $(1 \times d_{ff} \times d_{\text{model}})$ tensor to obtain a ciphertext that encrypts a $(1 \times L \times d_{\text{model}})$ tensor.
- (16) **Second Residual Connection:** The server sums both the output of the second feedforward layer and the first layer normalization to calculate the second residual connection (o) . Both ciphertexts are represented by tricyclic encodings of $(1 \times L \times d_{\text{model}})$ tensors.
- (17) **Second Layer Normalization:** The server calculates the second row-wise layer normalization (p) . This is the last operation in a transformer block, which outputs a $(1 \times L \times d_{\text{model}})$ tensor.

Our protocol solves the numerous challenges in enabling private LLM inference. Specifically, we overcome ciphertext capacity limits by breaking up matrices into matrix blocks. Tricycle leverages tricyclic encodings to enable efficient matrix multiplications, parallelize multiple attention heads, provide a free transpose, maintain packing consistency of residual connections, and support row-wise nonlinear operations. Since our matrix multiplications have optimal multiplicative depth of 1, we minimize the number of bootstrapping operations required. Finally, Tricycle uses several known approximations to nonlinear functions while proposing an improvement to the softmax approximation.

9 EVALUATION

Our evaluation includes two main sets of experiments: (1) microbenchmarks of our matrix multiplication algorithms and (2) end-to-end experiments with the Tricycle system. The microbenchmarks assess the efficiency of our matrix multiplication compared to other recent algorithms. The end-to-end experiments demonstrate Tricycle’s accuracy and efficiency when performing private inference on a Bert-Tiny model [10, 16]. We compare Tricycle to Rovida *et al.*’s CPU-based Bert-Tiny system [25], the only other open-source system that preserves the standard transformer architecture. In contrast, other works either significantly modify the transformer model (Rho *et al.* [24], Powerformer [23]), lack a full

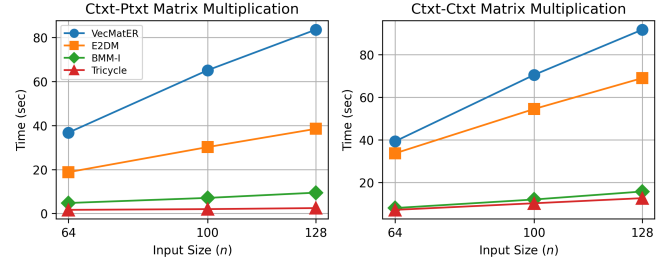


Figure 3: Microbenchmarks for $(n \times n) \times (n \times n)$ ciphertext-plaintext and ciphertext-ciphertext matrix multiplications.

end-to-end implementation (NEXUS [26]), or rely on GPUs to scale to larger models (Thor [21], Powerformer [23], Castro *et al.* [9]).

9.1 Implementation

We implemented Tricycle and all of our algorithms using OpenFHE version 1.2.3 [2], which provides the underlying cryptography primitives for CKKS, specifically the CKKS-RNS variant [6]. Our codebase also implements a number of other optimizations, including:

- Hoisting optimization [12]
- Reusing rotations for block matrix multiplications and for calculating V and K
- Optimized BSGS parameters for block matrix multiplications (since more input blocks means more baby steps while more output blocks means more giant steps)
- Reordered multiplications to minimize multiplicative depth
- Lazy relinearization for ciphertext-ciphertext matrix multiplications [21]
- Parallelization

For the Bert-Tiny model, we use a pre-trained model [16] and fine-tuned it on the Stanford Sentiment Treebank (SST) dataset.

9.2 Experimental Setup

All experiments in this work are conducted with 128-bit security with ring dimensions of 2^{16} , multiplicative depth of 29 (with 11 multiplicative depth remaining after bootstrapping), following the latest guidelines for parameter selection in the CKKS scheme and its underlying Ring-Learning-with-Errors (RLWE) assumption [1]. To enable a fair and meaningful comparison, we adapted the implementation of Rovida *et al.* [25] to use the same cryptographic parameters. All of the experiments were conducted on a x2iedn.32xlarge machine, which has 128 CPU cores and 4 terabytes of memory and take advantage of the parallelism implemented in OpenFHE.

9.3 Microbenchmarks

Figure 3 compares the performance of our batch matrix multiplications against VecMATER [25], E2DM’s matrix multiplications (proposed by Jiang *et al.* [15], and Chen *et al.*’s BMM-I [3]. E2DM’s matrix multiplications are extended in Powerformer [23] and Rho *et al.* [24] for their respective private LLM systems, while Rovida *et al.* use VecMATER directly in theirs [25]. All ciphertexts are configured with 11 levels of remaining multiplicative depth to reflect post-bootstrapping conditions. Our microbenchmarks are single-threaded at the application level but leverage OpenFHE’s multi-threaded backend. Both BMM-I and our method pad matrices to

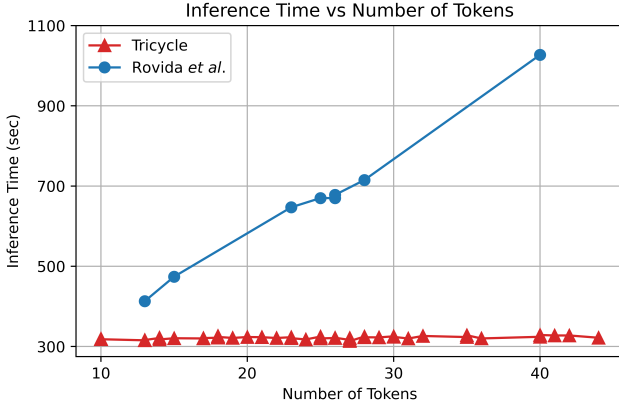


Figure 4: End-to-end private inference comparison between Tricycle and Rovida *et al.* [25] of a Bert-Tiny model.

coprime dimensions and apply appropriate block matrix partitioning. BMM-I includes the hoisting optimization [12] which was not originally proposed in their paper. Among all approaches, our matrix multiplications deliver the best performance. A primary distinction from BMM-I is our incorporation of the BSGS optimization, which BMM-I omits.

9.4 End-to-End Experiments

Next, we present the evaluation results of end-to-end experiments. Table 3 provides a breakdown of the average runtime for each operations. Bootstrapping emerges as the dominant bottleneck, accounting for approximately 72% of the entire execution time. In Tricycle, each transformer block has 5 bootstrapping operations where 4 are placed before and after the two layer normalization operations and one is placed inside the softmax operation. This substantial cost highlights the critical need to minimize the multiplicative depth, where each additional level of multiplicative depth in a deep neural network can trigger extra bootstrapping, exacerbating runtime overhead. Tricycle’s optimized matrix multiplications are designed to mitigate this issue, consuming only 7 levels of multiplicative depth, while the remaining 45 levels in a transformer block are dedicated to polynomial approximations of nonlinear operations such as GELU, softmax, and layer normalization. As a result, Tricycle effectively minimizes the multiplicative depth required for matrix multiplications in transformer models, ensuring efficiency without sacrificing accuracy.

Our evaluation shows that the error introduced by Tricycle’s Bert-Tiny model is minimal. Specifically, we observed an L_1 norm of approximately 0.011 between the plaintext model’s and the encrypted model’s outputs for 47 different input strings. This error is small enough that it rarely alters the model’s predictions, as demonstrated by examples of real inputs to Tricycle in Table 4.

Figure 4 compares the private inference time of Tricycle against Rovida *et al.* [25]. While the inference time in Rovida *et al.* grows

Operation	Time (sec)
Q	5.0
Reshape	2.5
K and V	7.7
Attention Score	10.7
Softmax	8.0
Softmax $\times V$	6.2
Output Projection	2.1
Layer Norm 1	8.2
Feedforward Layer 1	13.2
GELU	4.7
Feedforward Layer 2	5.0
Layer Norm 2	9.0
Bootstrapping	234.6
Total	322.0

Table 3: Time breakdown in Tricycle’s BERT-Tiny. The average token count was 26.1 and padded to 26.6.

String Input	True Logits	Tricycle Logits
it’s slow – very , very slow .	[1.667, -1.524]	[1.668, -1.527]
we root for (clara and paul) , even like them , though perhaps it’s an emotion closer to pity .	[-0.498, 0.401]	[-0.491, 0.390]
a tender , heartfelt family drama .	[-2.195, 1.859]	[-2.196, 1.860]

Table 4: Examples of input and output for a nonprivate and private (Tricycle) BERT-Tiny model. The larger logit indicates the predicted sentiment: the first for negative, the second for positive.

linearly with the number of tokens, Tricycle’s inference time remains largely stable. At 13 tokens, Tricycle is already $1.5\times$ faster and by around 40 tokens, it has a $3\times$ speedup. Tricycle’s performance gains stem primarily from more efficient matrix multiplications with reduced multiplicative depth. Both Tricycle and Rovida *et al.* use 5 bootstrapping operations per transformer block. However, Rovida *et al.* sometimes incurs additional multiplicative depth due to ciphertext data reorganization operations, such as their WRAPUPEXPANDED, WRAPUPREPEATED, and UPWRAPEXPANDED algorithms. To offset this, they avoid multiplicative depth elsewhere by replacing layer normalization with learned constants during training. While this substitution saves around 18 multiplicative depth per transformer block, reducing the number of bootstrapping operations by 2–3 per block compared to a typical polynomial approximation approach, it requires retraining and compromises model accuracy. In contrast, Tricycle preserves the original model architecture by using polynomial approximations for layer normalization, even at the cost of additional multiplicative depth. Despite this performance penalty, Tricycle remains more efficient than Rovida *et al.*’s system, enabling private LLM inference without retraining.

10 CONCLUSION

In this paper, we introduced Tricycle, a system for enabling private inference of transformer models under homomorphic encryption. Tricycle overcomes key challenges in private LLM inference by leveraging tricyclic encodings. We further propose a novel statistical max estimation technique for stabilizing softmax. Our matrix multiplications and end-to-end Tricycle system are implemented in OpenFHE, achieving matrix multiplications that consume just 7 levels of multiplicative depth per transformer block. Experimental results demonstrate that our matrix multiplication routines outperform prior work, and our end-to-end system achieves $1.5\times$ to $3\times$ speedups over Rovida *et al.*’s implementation. While our current work focuses on smaller-scale models, future GPU acceleration could enable Tricycle to scale to larger models.

REFERENCES

- [1] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. 2019. Homomorphic Encryption Standard. Cryptology ePrint Archive, Paper 2019/939. <https://eprint.iacr.org/2019/939>
- [2] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. Cryptology ePrint Archive, Paper 2022/915. <https://eprint.iacr.org/2022/915>
- [3] Jingwei Chen, Linhan Yang, Wenyuan Wu, Yang Liu, and Yong Feng. 2024. Homomorphic Matrix Operations under Bicyclic Encoding. Cryptology ePrint Archive, Paper 2024/1762. <https://eprint.iacr.org/2024/1762>
- [4] E. W. Cheney. 1966. *Introduction to approximation theory*. McGraw-Hill Book Co., New York.
- [5] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 360–384.
- [6] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. A full RNS variant of approximate homomorphic encryption. In *International Conference on Selected Areas in Cryptography*. Springer, 347–368.
- [7] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*, Tsuyoshi Takagi and Thomas Peyrin (Eds.). Springer International Publishing, Cham, 409–437.
- [8] Wonhee Cho, Guillaume Hanrot, Taeseong Kim, Minje Park, and Damien Stehlé. 2024. Fast and Accurate Homomorphic Softmax Evaluation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (CCS '24). Association for Computing Machinery, New York, NY, USA, 4391–4404. <https://doi.org/10.1145/3658644.3670369>
- [9] Leo de Castro, Antigoni Polychroniadou, and Daniel Escudero. 2024. Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption. In *Neurips Safe Generative AI Workshop 2024*.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] <https://arxiv.org/abs/1810.04805>
- [11] Robert E. Goldschmidt. 1964. *Applications of Division by Convergence*. Master's thesis. Massachusetts Institute of Technology.
- [12] Shai Halevi and Victor Shoup. 2014. Algorithms in helib. In *Advances in Cryptology – CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2014, Proceedings, Part I 34*. Springer, 554–571.
- [13] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. Iron: private inference on transformers. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1143, 14 pages.
- [14] Xiaoyang Hou, Jian Liu, Jingyu Li, Yuhao Li, Wenjie Lu, Cheng Hong, and Kui Ren. 2023. CipherGPT: Secure Two-Party GPT Inference. Cryptology ePrint Archive, Paper 2023/1147. <https://eprint.iacr.org/2023/1147>
- [15] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. 2018. Secure Outsourced Matrix Computation and Application to Neural Networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). Association for Computing Machinery, New York, NY, USA, 1209–1222. <https://doi.org/10.1145/3243734.3243837>
- [16] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. arXiv:1909.10351 [cs.CL] <https://arxiv.org/abs/1909.10351>
- [17] Gautam Kamath. 2015. Bounds on the expectation of the maximum of samples from a gaussian. http://www.gautamkamath.com/writings/gaussian_max.pdf
- [18] Yang Li, Xinyu Zhou, Yitong Wang, Liangxin Qian, and Jun Zhao. 2024. A Survey on Private Transformer Inference. arXiv:2412.08145 [cs.CR] <https://arxiv.org/abs/2412.08145>
- [19] Zhengyi Li, Kang Yang, Jin Tan, Wen-jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, et al. 2024. Nimbus: Secure and Efficient Two-Party Inference for Transformers. *Advances in Neural Information Processing Systems* 37 (2024), 21572–21600.
- [20] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. 2023. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive* (2023).
- [21] Jungho Moon, Dongwoo Yoo, Xiaoqian Jiang, and Miran Kim. 2024. THOR: Secure Transformer Inference with Homomorphic Encryption. Cryptology ePrint Archive, Paper 2024/1881. <https://eprint.iacr.org/2024/1881>
- [22] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. 2024. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4753–4771.
- [23] Dongjin Park, Eunsang Lee, and Joon-Woo Lee. 2024. Powerformer: Efficient Privacy-Preserving Transformer with Batch Rectifier-Power Max Function and Optimized Homomorphic Attention. Cryptology ePrint Archive, Paper 2024/1429. <https://eprint.iacr.org/2024/1429>
- [24] Donghwan Rho, Taeseong Kim, Minje Park, Jung Woo Kim, Hyunsik Chae, Ernest K. Ryu, and Jung Hee Cheon. 2025. Encryption-Friendly LLM Architecture. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=pbre0HKsfE>
- [25] Lorenzo Roveda and Alberto Leporati. 2024. Transformer-based Language Models and Homomorphic Encryption: An Intersection with BERT-tiny. In *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics* (Porto, Portugal) (IWSPA '24). Association for Computing Machinery, New York, NY, USA, 3–13. <https://doi.org/10.1145/3643651.3659893>
- [26] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wenjie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. 2024. Secure Transformer Inference Made Non-interactive. Cryptology ePrint Archive, Paper 2024/136. <https://eprint.iacr.org/2024/136>
- [27] Mengxin Zheng, Qian Lou, and Lei Jiang. 2023. Primer: Fast private transformer inference on encrypted data. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

A CHINESE REMAINDER THEOREM

Let $m, n \in \mathbb{Z}_{>0}$ such that $\gcd(m, n) = 1$, and let $a, b \in \mathbb{Z}$. The system of congruences

$$\begin{cases} x \equiv a \pmod{m} \\ x \equiv b \pmod{n} \end{cases}$$

has a unique solution modulo mn which can be explicitly constructed as

$$x = (a + m \cdot (m^{-1} \pmod{n}) \cdot (b - a)) \pmod{mn}.$$

$(m^{-1} \pmod{n})$ is an inverse modulo n such that $m \cdot m^{-1} \equiv 1 \pmod{n}$. Symmetrically, x can be constructed as

$$x = (b + n \cdot (n^{-1} \pmod{m}) \cdot (a - b)) \pmod{mn}.$$

The Chinese Remainder Theorem can generalize to systems of three or more congruences. The system of n congruences

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \dots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

with $\gcd(m_1, m_2, \dots, m_n) = 1$ has a unique solution modulo M where $M = \prod_{i=1}^n m_i$, which can be expressed as

$$x \pmod{M} = \left(\sum_{i=1}^n r_i \cdot \frac{M}{m_i} \cdot \left(\left(\frac{M}{m_i} \right)^{-1} \pmod{m_i} \right) \right) \pmod{M}$$

where M/m_i is the product of all $m_1, m_2, \dots, m_n$ except m_i .

B FREE TRANSPOSE THEOREM

We prove that for any n -cyclic encoding of an n -dimensional tensor, all transposes are provided for free.

Definition 1: Let $d_1, d_2, \dots, d_n \in \mathbb{N}$. An n -dimensional tensor is defined by

$$A = [a_{i_1, i_2, \dots, i_n}] \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}.$$

Definition 2: Assume that $\gcd(d_1, d_2, \dots, d_n) = 1$. The n -cyclic encoding of A is defined by the mapping

$$\varphi: \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n} \rightarrow \mathbb{R}^{\prod_{i=1}^n d_i},$$

given by

$$\varphi(A)_k = a_{k \pmod{d_1}, k \pmod{d_2}, \dots, k \pmod{d_n}},$$

for all $k \in \mathbb{Z}_{\prod_{i=1}^n d_i}$.

Definition 3: A transpose of A is defined by choosing a new ordering of its indices. That is, if we reassign the indices according to

$$\tau: (1, 2, \dots, n) \rightarrow (a_1, a_2, \dots, a_n),$$

where $(a_1, a_2, \dots, a_n)$ is a permutation of $(1, 2, \dots, n)$ then the transposed tensor A^τ has components

$$\begin{aligned} A^\tau &= [a_{i_1, i_2, \dots, i_n}^\tau] \\ &= [a_{i_{\tau(1)}, i_{\tau(2)}, \dots, i_{\tau(n)}}] \in \mathbb{R}^{d_{\tau(1)} \times d_{\tau(2)} \times \dots \times d_{\tau(n)}}. \end{aligned} \quad (30)$$

The inverse of the transpose is defined as

$$\begin{aligned} A &= (A^\tau)^{\tau^{-1}} = [a_{i_1, i_2, \dots, i_n}] = [a_{i_{\tau^{-1}(1)}, i_{\tau^{-1}(2)}, \dots, i_{\tau^{-1}(n)}}^{\tau^{-1}}] \\ &= [a_{i_{\tau^{-1}(\tau(1))}, i_{\tau^{-1}(\tau(2))}, \dots, i_{\tau^{-1}(\tau(n))}}^\tau] \\ &= [a_{i_{\tau^{-1}(\tau(1))}, i_{\tau^{-1}(\tau(2))}, \dots, i_{\tau^{-1}(\tau(n))}}]. \end{aligned} \quad (31)$$

Theorem: For an n -dimensional tensor A with dimensions $(d_1 \times d_2 \times \dots \times d_n)$ and for any transpose A^τ defined by a permutation $(a_1, a_2, \dots, a_n)$, then $\varphi(A^\tau) = \varphi(A)$.

Starting with the definition of n -cyclic encoding of A^τ , we get

$$\begin{aligned} \varphi(A^\tau)_k &= a_{k \pmod{d_{\tau(1)}}, k \pmod{d_{\tau(2)}}, \dots, k \pmod{d_{\tau(n)}}}^\tau \\ &\quad \text{for all } k \in \mathbb{Z}_{\prod_{i=1}^n d_i}. \end{aligned} \quad (32)$$

Using the inverse of the transpose, we get

$$= a_{k \pmod{d_1}, k \pmod{d_2}, \dots, k \pmod{d_n}} = \varphi(A)_k \quad (33)$$

which proves the theorem.

C BATCH MATRIX MULTIPLICATION

C.1 Broadcasted Batch Matrix Multiplication

For a $(1 \times m \times n)$ tensor A and an $(h \times n \times p)$ tensor B , the broadcasted batch matrix multiplication is expressed as

$$C_{i,j,k} = \left[\sum_{c=0}^{n-1} a_{0,j,c} \cdot b_{i,c,k} \right] \in \mathbb{R}^{h \times m \times p}. \quad (34)$$

Applying the tricyclic encoding, we get

$$\begin{aligned} \varphi(Z)_k &= \sum_{c=0}^{n-1} \left(a_{0, k \pmod{m}, (k+c) \pmod{n}} \cdot \right. \\ &\quad \left. b_{k \pmod{h}, (k+c) \pmod{n}, k \pmod{p}} \right). \end{aligned} \quad (35)$$

By applying the Chinese Remainder Theorem and definition of tricyclic encoding, we get

$$\begin{aligned} &= \sum_{c=0}^{m-1} \left(\varphi(A)_{(k+cm(m^{-1} \pmod{n})) \pmod{mn}} \odot \right. \\ &\quad \left. \varphi(B)_{(k+chp((hp)^{-1} \pmod{n})) \pmod{hnp}} \right). \end{aligned} \quad (36)$$

Finally, converting to rotations, we get

$$\begin{aligned} \varphi(Z) &= \sum_{c=0}^{n-1} \left(\varphi \left(\varphi(A), cm(m^{-1} \pmod{n}) \pmod{mn} \right) \odot \right. \\ &\quad \left. \varphi \left(\varphi(B), chp((hp)^{-1} \pmod{n}) \pmod{hnp} \right) \right). \end{aligned} \quad (37)$$

This shows that we can compute the broadcasted batch matrix multiplication by computing rotations of every $m(m^{-1} \pmod{n})$ and $hp((hp)^{-1} \pmod{n})$ for $\varphi(A)$ and $\varphi(B)$, respectively, and summing their product.

To show the ciphertext-plaintext broadcasted batch matrix multiplication between a ciphertext A and plaintext B , we can start with Equation (35) and simplify A to

$$\begin{aligned} &= \sum_{c=0}^{m-1} \left(\varphi(A)_{(k+cm(m^{-1} \pmod{n})) \pmod{mn}} \odot \right. \\ &\quad \left. b_{k \pmod{h}, (k+c) \pmod{n}, k \pmod{p}} \right). \end{aligned} \quad (38)$$

A can now be converted rotations and B to a plaintext vector as follows:

$$\varphi(Z) = \sum_{c=0}^{n-1} \left(\rho \left(\varphi(A), cm(m^{-1} \bmod n) \right) \bmod (mn) \right) \odot \text{BroadcastDiag}(B, c) \quad (39)$$

where $\text{BroadcastDiag}(B, c) = b_{k \bmod h, (k+c) \bmod n, k \bmod p}$.

C.2 Concatenated Matrix Multiplication

For an $(h \times m \times n)$ ciphertext tensor A and an $(h \times n \times p)$ plaintext tensor B , the concatenated matrix multiplication is expressed as

$$C_{0,j,k} = \left[\sum_{d=0}^{h-1} \sum_{c=0}^{n-1} a_{d,j,c} \cdot b_{d,c,k} \right] \in \mathbb{R}^{1 \times m \times p}. \quad (40)$$

We can factor out the inner summation into C' as follows:

$$C_{0,j,k} = \sum_{d=0}^{h-1} C'_{d,j,k} \quad \text{where } C'_{d,j,k} = \left[\sum_{c=0}^{n-1} a_{d,j,c} \cdot b_{d,c,k} \right] \in \mathbb{R}^{h \times m \times p} \quad (41)$$

First, we will simplify C' . Note that the equation for C' is exactly the same as a batch matrix multiplication. Using a similar process, we get

$$\varphi(C')_{k'} = \sum_{c=0}^{n-1} \left(a_{k' \bmod h, k' \bmod m, (k'+c) \bmod n} \cdot b_{k' \bmod h, (k'+c) \bmod n, k' \bmod p} \right). \quad (42)$$

We remark that the d , j , and k in $C'_{d,j,k}$ became substituted for $k' \bmod h$, $k' \bmod m$, and $k' \bmod p$, respectively.

We return to simplifying C . We first apply the definition of tricyclic encoding to C .

$$\varphi(Z)_k = \sum_{d=0}^{h-1} c'_{d, k \bmod m, k \bmod p} \quad (43)$$

$$= \sum_{d=0}^{h-1} c'_{(k+d) \bmod h, k \bmod m, k \bmod p}. \quad (44)$$

This means that to calculate the k -th value of $\varphi(Z)$, we want to sum up values from C' . These values are obtained from the $((k+d) \bmod h)$ -th matrix of C' , the $(k \bmod m)$ -th row of that matrix, and $(k \bmod p)$ -th column of that row. We can now substitute C' from Equation (42) back into Equation (43). We can substitute $(k+d) \bmod h$ in for the first dimension of $C'_{d,j,k}$ and likewise for the rest of the dimensions as follows

$$\varphi(Z)_k = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(a_{(k+d) \bmod h, k \bmod m, (k+c) \bmod n} \cdot b_{(k+d) \bmod h, (k+c) \bmod n, k \bmod p} \right). \quad (45)$$

Using CRT, we can find that $x = (d \cdot (mn) \cdot ((mn)^{-1} \bmod h) + c \cdot (hm) \cdot ((hm)^{-1} \bmod n)) \bmod (hmn)$ is a solution to the congruence

$x \equiv d \bmod h$, $x \equiv 0 \bmod m$, and $x \equiv c \bmod n$. This means we can simplify to

$$\varphi(Z)_k = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(a_{(k+x) \bmod h, (k+x) \bmod m, (k+x) \bmod n} \cdot b_{(k+d) \bmod h, (k+c) \bmod n, k \bmod p} \right). \quad (46)$$

Now, we apply the tricyclic encoding to A and simplify to

$$\varphi(Z)_k = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(\varphi(A)_{(k+d \cdot r_1 + c \cdot r_2) \bmod (h \cdot m \cdot n)} \cdot b_{(k+d) \bmod h, (k+c) \bmod n, k \bmod p} \right) \quad \text{where } r_1 = m \cdot n \cdot ((m \cdot n)^{-1} \bmod h),$$

$$r_2 = h \cdot m \cdot ((h \cdot m)^{-1} \bmod n). \quad (47)$$

We can likewise take the same steps for B . Using CRT, we can find that $y = (d \cdot (np) \cdot ((np)^{-1} \bmod h) + c \cdot (hp) \cdot ((hp)^{-1} \bmod n)) \bmod (hnp)$ is a solution to the congruence $y \equiv d \bmod h$, $y \equiv c \bmod n$, and $y \equiv 0 \bmod p$. We use this and apply the definition of tricyclic encoding to simplify to

$$\varphi(Z)_k = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(\varphi(A)_{(k+d \cdot r_1 + c \cdot r_2) \bmod (h \cdot m \cdot n)} \cdot \varphi(B)_{(k+y) \bmod (h \cdot n \cdot p)} \right). \quad (48)$$

We can finally convert to using rotations and show that the equation for a ciphertext-ciphertext matrix multiplication is as follows:

$$\varphi(Z) = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(\rho(\varphi(A), (d \cdot r_1 + c \cdot r_2) \bmod (h \cdot m \cdot n)) \odot \rho(\varphi(B), (d \cdot r_3 + c \cdot r_4) \bmod (h \cdot n \cdot p)) \right) \quad \text{where } r_1 = m \cdot n \cdot ((m \cdot n)^{-1} \bmod h),$$

$$r_2 = h \cdot m \cdot ((h \cdot m)^{-1} \bmod n),$$

$$r_3 = n \cdot p \cdot ((n \cdot p)^{-1} \bmod h),$$

$$r_4 = h \cdot p \cdot ((h \cdot p)^{-1} \bmod n). \quad (49)$$

Discussion on this equation can be found in Section 5.2.3.

For a ciphertext-plaintext concatenated matrix multiplication between a ciphertext A and plaintext B , we can derive the diagonals required by converting from Equation (47) to

$$\varphi(Z) = \sum_{d=0}^{h-1} \sum_{c=0}^{n-1} \left(\rho(\varphi(A), (d \cdot r_1 + c \cdot r_2) \bmod (h \cdot m \cdot n)) \odot \text{ConcatDiag}(B, c, d) \right) \quad \text{where } \forall k \in \mathbb{Z}_{m \cdot p}, \text{ConcatDiag}(B, c, d)_k =$$

$$b_{(k+d) \bmod h, (k+c) \bmod n, k \bmod p}. \quad (50)$$

d is a summation index that iterates over matrices in A and B . c is a summation index that iterates columns and rows in those matrices in A and B , respectively. r_1 represents the rotation amount required to obtain the next matrix in the $\varphi(A)$ while r_2 represents the rotation amount required to obtain the next column in the same matrix. Finally, $\text{ConcatDiag}(B, c, d)$ is the function to calculate the plaintext needed to multiply by the rotated ciphertext to obtain the correct output.

D BABY-STEP GIANT-STEP CIPHERTEXT-PLAINTEXT MATRIX MULTIPLICATIONS

We prove without loss of generality to broadcasted batch matrix multiplication and concatenated batch matrix multiplication that an $(h \times m \times n)$ ciphertext X multiplied by an $(h \times n \times p)$ plaintext Y under tricyclic encoding can apply BSGS by showing that it can be expressed in the form $\sum_{c=0}^{N-1} \rho(x, T \cdot c) \odot y$ with periodicity $T = h \cdot m$. Beginning with Equation (15), which shows ciphertext-plaintext batch matrix multiplications under tricyclic encoding, we have

$$\varphi(Z) = \sum_{c=0}^{n-1} (\rho(\varphi(X), (chm((hm)^{-1} \bmod n)) \bmod (hmn)) \odot \text{BatchDiag}(Y, c)) \quad (51)$$

where $\text{BatchDiag}(Y, c)_k = y_{k \bmod h, (k+c) \bmod n, k \bmod p}$ for $k \in \mathbb{Z}_{hmp}$. To prove that BSGS is applicable, we can reindex c through a series of steps as follows:

$$= \sum_{c=0}^{n-1} \rho(\varphi(X), (chm((hm)^{-1} \bmod n)) \bmod (hmn)) \odot \text{BatchDiag}(Y, c) \quad (52)$$

$$= \sum_{c=0}^{n-1} (\rho(\varphi(X), ((c \bmod n) \cdot ((hm)^{-1} \bmod n) \cdot hm) \bmod (hmn)) \odot \text{BatchDiag}(Y, c \bmod n)) \quad (53)$$

$$= \sum_{c=0}^{n-1} (\rho(\varphi(X), ((c \cdot (hm)^{-1} \bmod n) \cdot hm) \bmod (hmn)) \odot \text{BatchDiag}(Y, c \cdot (hm)^{-1} \cdot hm \bmod n)) \quad (54)$$

Since h, m , and n are coprime, multiplying each element of $\{0, 1, \dots, n-1\}$ by hm or its inverse modulo n simply permutes the set, obtaining the same numbers in a different order. This allows us to reindex c to $c' = (c \cdot (hm)^{-1} \bmod n)$, which becomes

$$= \sum_{c'=0}^{n-1} (\rho(\varphi(X), (c' \cdot hm) \bmod (hmn)) \odot \text{BatchDiag}(Y, (c' \cdot hm) \bmod n)) \quad (55)$$

$$= \sum_{c'=0}^{n-1} (\rho(\varphi(X), c' \cdot hm) \odot \text{BatchDiag}'(Y, c')) \quad (56)$$

where $\text{BatchDiag}'(Y, c')_k = y_{k \bmod h, (k+c' \cdot hm) \bmod n, k \bmod p}$ for $k \in \mathbb{Z}_{hmp}$. This simplification means that BSGS can be applied for $x = \varphi(A)$, $y = \text{BatchDiag}'(Y, c)$, and periodicity $T = hm$.

E BABY-STEP GIANT-STEP FOR CIPHERTEXT-CIPHERTEXT MATRIX MULTIPLICATIONS

We prove without loss of generality to broadcasted batch matrix multiplication and concatenated batch matrix multiplication that

an $(h \times m \times n)$ ciphertext A multiplied by an $(h \times n \times p)$ ciphertext B under tricyclic encoding can apply BSGS by showing that it can be expressed in the form $\sum_{c=0}^{N-1} \rho(x, T \cdot c) \odot y$ with periodicity $T = h \cdot m$. Beginning with Equation (9), which shows ciphertext-plaintext batch matrix multiplications under tricyclic encoding, we have

$$\varphi(Z) = \sum_{c=0}^{n-1} (\rho(\varphi(A), chm((hm)^{-1} \bmod n)) \bmod (hmn)) \odot \rho(\varphi(B), (chp((hp)^{-1} \bmod n)) \bmod (hnp)) \quad (57)$$

To prove that BSGS is applicable, we can reindex c through a series of steps as follows:

$$\varphi(Z) = \sum_{c=0}^{n-1} (\rho(\varphi(A), (c \bmod n) \cdot ((hm)^{-1} \bmod n) \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), (chp((hp)^{-1} \bmod n)) \bmod (hnp))) \quad (58)$$

$$= \sum_{c=0}^{n-1} (\rho(\varphi(A), (c(hm)^{-1} \bmod n) \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), (chp((hp)^{-1} \bmod n)) \bmod (hnp))) \quad (59)$$

$$= \sum_{c=0}^{n-1} (\rho(\varphi(A), (c(hm)^{-1} \bmod n) \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), ((c(hm)^{-1} \cdot hm \bmod n) \cdot hp((hp)^{-1} \bmod n)) \bmod (hnp))) \quad (60)$$

Similar to applying BSGS to ciphertext-plaintext matrix multiplications, h, m , and n are coprime, so multiplying each element of $\{0, 1, \dots, n-1\}$ by hm or its inverse modulo n simply permutes the set, obtaining the same numbers in a different order. This allows us to reindex c to $c' = (c \cdot (hm)^{-1} \bmod n)$, which becomes

$$= \sum_{c'=0}^{n-1} (\rho(\varphi(A), (c' \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), ((c' \cdot hm \bmod n) \cdot hp((hp)^{-1} \bmod n)) \bmod (hnp))) \quad (61)$$

$$= \sum_{c'=0}^{n-1} (\rho(\varphi(A), (c' \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), (hp(c' \cdot hm(hp)^{-1} \bmod n)) \bmod (hnp))) \quad (62)$$

$$\varphi(Z) = \sum_{c'=0}^{n-1} (\rho(\varphi(A), (c' \cdot hm \bmod (hmn)) \odot \rho(\varphi(B), (hp(c' \cdot m(p)^{-1} \bmod n)) \bmod (hnp))) \quad (63)$$

This simplification means that BSGS can be applied for $x = \varphi(A)$, $y = \varphi(B)$, periodicity $T = hm$, and $\text{RotY}(c') = (hp(c' \cdot m(p)^{-1} \bmod n)) \bmod (hnp)$.

F ROW-WISE SUMMATION UNDER TRICYCLIC ENCODING

Row-wise summations are used in softmax and layer normalization operations. In softmax, our statistical max estimation, described in Section 7.1, requires two row-wise summations for calculating the mean and variance and the softmax itself requires calculating a sum of exponentials, a third row-wise summation. Layer normalization requires two row-wise summations for mean and variance. The row-wise summation algorithms are described in Algorithms 1 and 2, where Algorithm 1 is used when only half or fewer of the ciphertext slots are used while Algorithm 2 is otherwise used.

Algorithm 1 Row-wise Summation under Tricyclic Encoding. This algorithm assumes that no more than half of available slots are used. $\lceil \log_2(n) \rceil + 1$ homomorphic rotations and additions are required.

Input: Ciphertext c encrypting a tricyclically encoded tensor with dimensions $(h \times m \times n)$ and zero padding.

Output: Ciphertext encrypting a tricyclically encoded tensor with dimensions $(h \times m \times n)$ containing row-wise sums of c , no additional multiplicative depth, and without zero padding.

Prerequisite: $h \cdot m \cdot n \leq (\text{number of slots})/2$

```

1:  $\ell \leftarrow \lceil \log_2(n) \rceil$ 
2:  $\text{sum} \leftarrow c$ 
3: for  $i = 0$  to  $\ell - 1$  do
4:    $\text{sum} \leftarrow \text{sum} + \rho(\text{sum}, h \cdot m \cdot 2^i)$ 
5: end for
6: return  $\text{sum} + \rho(\text{sum}, -h \cdot m \cdot 2^\ell)$ 

```

Algorithm 2 Row-wise Summation under Tricyclic Encoding. This is less efficient (requires more rotations and one extra multiplicative depth) than Algorithm 1, so it is only used when more than half of the available slots are used, e.g., in layer normalization.

Input: Ciphertext c encrypting a tricyclically encoded tensor of dimensions $(h \times m \times n)$.

Output: Ciphertext of dimensions $(h \times m \times n)$ containing row-wise sums of c , with zero padding and one additional multiplicative depth.

```

1:  $\ell \leftarrow \lceil \log_2(n) \rceil + 1$ 
2:  $u \leftarrow c$ 
3: Initialize list  $U$  with  $U[0] \leftarrow u$ 
4: for  $i = 1$  to  $\ell - 1$  do
5:    $u \leftarrow u + \rho(u, h \cdot m \cdot 2^{i-1})$ 
6:    $U[i] \leftarrow u$ 
7: end for
8:  $s \leftarrow U[\ell - 1]$ 
9: for  $i = 0$  to  $\ell - 2$  do
10:  if  $\lfloor \frac{n}{2^i} \rfloor \bmod 2 = 1$  then
11:     $s \leftarrow s + \rho(U[i], h \cdot m \cdot (n - (n \bmod 2^{i+1})))$ 
12:  end if
13: end for
14:  $\text{mask} \leftarrow$  Plaintext vector with 1 for  $0 \leq i < h \cdot m$ , else 0
15:  $v \leftarrow s \odot \text{mask}$ 
16: Initialize list  $V$  with  $V[0] \leftarrow v$ 
17: for  $i = 1$  to  $\ell - 1$  do
18:   $v \leftarrow v + \rho(v, -h \cdot m \cdot 2^{i-1})$ 
19:   $V[i] \leftarrow v$ 
20: end for
21:  $t \leftarrow V[\ell - 1]$ 
22: for  $i = 0$  to  $\ell - 2$  do
23:  if  $\lfloor \frac{n}{2^i} \rfloor \bmod 2 = 1$  then
24:     $t \leftarrow \rho(t, -h \cdot m \cdot 2^i) + V[i]$ 
25:  end if
26: end for
27: return  $t$ 

```

G FOLDING RESHAPE WITH COMPUTING K AND V

We demonstrate that the reshape operation can be folded into the computation of K and V , allowing both to share a single multiplicative depth. This is theoretically appealing, as it removes the only additional multiplicative depth required for matrix multiplications. In practice, however, the gain is offset by an extra multiplicative depth later needed for zero padding in the softmax operation.

We begin with an input ciphertext X , which encrypts a tricyclically encoded $(1 \times L \times d_{\text{model}})$ tensor. The reshape operation is defined as $X' = I' \cdot X$ where I' is a $(1 \times (L+r) \times L)$ tensor which contains the tall identity matrix. Afterwards, K and V are calculated by computing $K = X' \cdot W_K$ and $V = X' \cdot W_V$ where W_K and W_V are represented by $(h \times d_{\text{model}} \times d_k)$ tensors. Without loss of generality to V , we show that reshaping and multiplying by W_K to compute K is as follows:

$$K = I \cdot X \cdot W_K = (X^\tau \cdot I^\tau)^{\tau^{-1}} \cdot W_K \quad (64)$$

X^τ is a $(1 \times d_{\text{model}} \times L)$ tensor and I^τ is a $(1 \times L \times (L+r))$ tensor. X is tricyclically encoded, so it can be transposed for free. We can now apply the ciphertext-plaintext batch matrix multiplication (Equation 56) between $X^\tau \cdot I^\tau$, we get

$$\left(\sum_{a=0}^{L-1} (\rho(\varphi(X), a \cdot d_{\text{model}}) \odot \text{BatchDiag}'(I^\tau, a)) \right)^\tau \cdot W_K \quad (65)$$

where $\text{BatchDiag}'(I^\tau, a)_u = i_{0, (u+a \cdot d_{\text{model}}) \bmod L, u \bmod (L+r)}^\tau$. We can transpose I^τ back to I to rewrite $\text{BatchDiag}'(I^\tau, a)$ in terms of I to get $\text{BatchDiag}'(I, a)_u = i_{0, u \bmod (L+r), (u+a \cdot d_{\text{model}}) \bmod L}$. Since the inverse transpose of $X^\tau \cdot I^\tau$ is provided for free by the tricyclic encoding (resulting in a $(1 \times (L+r) \times d_{\text{model}})$ tensor), we can now

multiply by W_K and similarly apply Equation 56 as follows:

$$= \sum_{b=0}^{d_{\text{model}}-1} \rho \left(\sum_{a=0}^{L-1} (\rho(\varphi(X), a \cdot d_{\text{model}}) \odot \text{BatchDiag}'(I, a), b \cdot (L+r)) \odot \text{BatchDiag}'(W_K, b) \right) \quad (66)$$

where the u -th value of $\text{BatchDiag}'(W_K, b)$ is equal to $w_{u \bmod h, (u+b \cdot (L+r)) \bmod d_{\text{model}}, u \bmod d_k}$. By distributing the outer rotation inside the summation, the equation simplifies to

$$= \sum_{b=0}^{d_{\text{model}}-1} \sum_{a=0}^{L-1} (\rho(\varphi(X), a \cdot d_{\text{model}} + b \cdot (L+r)) \odot \rho(\text{BatchDiag}'(I, a, b \cdot (L+r)) \cdot \text{BatchDiag}'(W_K, b))) \quad (67)$$

$$= \sum_{b=0}^{d_{\text{model}}-1} \sum_{a=0}^{L-1} (\rho(\varphi(X), a \cdot d_{\text{model}} + b \cdot (L+r)) \odot \text{ReshapeDiag}(I, a, b) \cdot \text{BatchDiag}'(W_K, b)) = \varphi(K) \quad (68)$$

where the u -th value of $\text{ReshapeDiag}(I, a, b)$, defined as $\text{ReshapeDiag}(I, a, b)_u$ is equal to $i_{0, u+b \cdot (L+r) \bmod (L+r), u+a \cdot d_{\text{model}}+b \cdot (L+r) \bmod L}$, simplifying to $i_{0, u \bmod (L+r), u+a \cdot d_{\text{model}}+b \cdot (L+r) \bmod L}$. This demonstrates that the two operations can be folded together because $\text{ReshapeDiag}(I, a, b) \cdot \text{BatchDiag}'(W_K, b)$ is a plaintext vector, so only one multiplicative depth is required.

While minimizing multiplicative depth is a key performance goal, applying this folding optimization incurs significant computational costs. Without folding, the operation requires approximately $2\sqrt{L} + 2\sqrt{d_{\text{model}}}$ rotations and $L + d_{\text{model}}$ homomorphic multiplications for two separate batch matrix multiplications. With folding (and after applying BSGS), the number of rotations increases to approximately $2\sqrt{L \cdot d_{\text{model}}}$ and the number of multiplications increases to $L \cdot d_{\text{model}}$. The benefit of reduced multiplicative depth is ultimately negated, however, as an additional multiplicative depth is later required to enforce zero padding for row-wise summation in the softmax.